

# PokerProject2: Commercial Behavioral Analytics from Gameplay Logs

## Executive Summary

This project converts imperfect PHH gameplay logs into a commercial analytics case study. The focus is not poker strategy; it is participant behavior, contest lifecycle depth, interaction intensity, resolution visibility, and outcome concentration.

The strongest finding is that action-level event data can support useful product analytics even when private-card visibility is incomplete. The notebook therefore emphasizes observable behavior and explicitly separates reliable event metrics from weaker card-strength or strategy claims.

In practical terms, the workflow shows how an analyst can audit messy data, define business-readable metrics, test data limitations, and translate gameplay logs into product and ecosystem insights.

## Business Question

How can raw gameplay logs be transformed into metrics that help a gaming, sportsbook, fantasy, or contest operator understand participant behavior and product experience?

This analysis focuses on five practical questions:

- How much interaction does a typical hand create?
- Which hands generate higher-commitment action?
- How far through the contest lifecycle do hands progress before resolving?
- Is outcome intensity concentrated in a small share of hands?
- How much of the underlying event data is visible enough to support stronger conclusions?

## Methodology and Data Flow

The notebook follows a simple analytics workflow:

1. Audit available files and separate single-hand `.phh` records from multi-hand `.phhs` containers.
2. Parse the single-hand PHH files into a flat hand-level table.
3. Derive observable behavioral metrics such as action depth, high-commitment action rate, lifecycle stage, resolution path, and outcome intensity.

4. Validate card visibility and obfuscation before making any claims that depend on private cards.
5. Translate the metrics into commercial takeaways that would make sense to non-poker stakeholders.

The `.phhs` HandHQ files are real multi-hand containers and can be read with `HandHistory.load_all()`. They are audited here but kept outside the main analysis so the primary dataset remains one row per single-hand file.

## Load PHH Files and Audit Source Coverage

The notebook expects the PHH files to live in the local ``data`` folder inside this project. The audit below confirms how many files are available by source folder before any behavioral analysis is run.

```
In [1]: from pathlib import Path
import re
import pandas as pd
import numpy as np
from tqdm import tqdm

# Install only if needed
try:
    from pokerkit import HandHistory
except ModuleNotFoundError:
    import sys
    !{sys.executable} -m pip install pokerkit
    from pokerkit import HandHistory

# Resolve the project folder relative to the notebook when possible.
PROJECT_ROOT = Path.cwd()
if not (PROJECT_ROOT / "data").exists():
    PROJECT_ROOT = Path.home() / "Downloads" / "poker-hand-histories"
PHH_ROOT = PROJECT_ROOT / "data"

candidate_files = sorted(
    path
    for path in PHH_ROOT.rglob("*")
    if path.is_file() and path.suffix.lower() in {".phh", ".phhs"}
)

# The core analysis below uses one row per single-hand .phh file. The HandHQ
# folder uses .phhs multi-hand containers, which PokerKit can read with
# HandHistory.load_all(). They are audited here but excluded from the main
# analysis to avoid mixing two different units of observation.
phh_files = sorted(path for path in candidate_files if path.suffix.lower() == ".phh")
phhs_files = sorted(path for path in candidate_files if path.suffix.lower() == ".phhs")

def source_folder(path):
    path = Path(path)
    rel = path.relative_to(PHH_ROOT)
```

```

        return rel.parts[0] if len(rel.parts) > 1 else "root_examples"

file_inventory = pd.DataFrame([
    {
        "source_folder": source_folder(path),
        "extension": path.suffix.lower(),
        "included_in_main_analysis": path.suffix.lower() == ".phh",
    }
    for path in candidate_files
])

dataset_audit = (
    file_inventory
    .groupby(["source_folder", "extension", "included_in_main_analysis"], as_index=False)
    .size()
    .rename(columns={"size": "files"})
    .sort_values(["included_in_main_analysis", "source_folder", "extension"], ascending=True)
)

print(PROJECT_ROOT)
print(PHH_ROOT)
print("Exists?", PHH_ROOT.exists())
print(f"Found {len(candidate_files):,} PHH/PHHS candidate files")
print(f"Using {len(phh_files):,} single-hand .phh files in the main analysis")
print(f"Excluding {len(phhs_files):,} .phhs multi-hand container files from the main analysis")

phhs_sample_hand_count = np.nan
if phhs_files:
    import warnings
    try:
        with warnings.catch_warnings():
            warnings.simplefilter("ignore")
            with open(phhs_files[0], "rb") as f:
                phhs_sample_hand_count = sum(1 for _ in HandHistory.load_all(f))
        print(f"Sample .phhs container loads {phhs_sample_hand_count:,.0f} hands via HandHistory.load_all()")
    except Exception as e:
        print(f"Sample .phhs load_all check failed: {e}")

display(dataset_audit)

```

C:\Users\J1999\Downloads\poker-hand-histories

C:\Users\J1999\Downloads\poker-hand-histories\data

Exists? True

Found 31,870 PHH/PHHS candidate files

Using 10,088 single-hand .phh files in the main analysis

Excluding 21,782 .phhs multi-hand container files from the main analysis

Sample .phhs container loads 999 hands via HandHistory.load\_all()

|   | source_folder | extension | included_in_main_analysis | files |
|---|---------------|-----------|---------------------------|-------|
| 1 | pluribus      | .phh      | True                      | 10000 |
| 2 | root_examples | .phh      | True                      | 5     |
| 3 | wsop          | .phh      | True                      | 83    |
| 0 | handhq        | .phhs     | False                     | 21782 |

## Reusable Parsing Helpers

The reusable layer below keeps parsing, card visibility checks, outcome extraction, participant-row creation, and obfuscation validation in one place. Later sections call these helpers instead of redefining the same loading and action-parsing logic.

```
In [2]: CARD_RE = re.compile(r"\b[2-9TJQKA][cdhs]\b", re.IGNORECASE)

def parse_path_metadata(path):
    path = Path(path)
    path_str = str(path).replace("\\", "/").lower()
    parts = list(path.parts)

    is_wsop = "/wsop/" in path_str
    folder_year = None
    event_id = None
    table_id = None

    if is_wsop:
        try:
            parts_lower = [p.lower() for p in parts]
            wsop_idx = parts_lower.index("wsop")
            folder_year = int(parts[wsop_idx + 1])
            event_id = parts[wsop_idx + 2] if len(parts) > wsop_idx + 2 else None
            table_id = parts[wsop_idx + 3] if len(parts) > wsop_idx + 3 else None
        except Exception:
            pass

    return {
        "is_wsop": is_wsop,
        "folder_year": folder_year,
        "event_id": event_id,
        "table_id": table_id,
    }

def load_hand(path):
    with open(Path(path), "rb") as f:
        return HandHistory.load(f)

def safe_len(x):
```

```

    try:
        return len(x)
    except Exception:
        return np.nan

def to_list(x):
    return list(x) if isinstance(x, (list, tuple)) else []

def numeric_values(x):
    return [v for v in to_list(x) if isinstance(v, (int, float))]

def safe_sum_numeric(x):
    vals = numeric_values(x)
    return sum(vals) if vals else np.nan

def get_attr(obj, name, default=None):
    return getattr(obj, name, default)

def hand_actions(hh):
    return [str(a).lower().strip() for a in to_list(get_attr(hh, "actions", []))]

def action_code(action):
    parts = str(action).lower().strip().split()
    if len(parts) >= 2 and parts[0].startswith("p"):
        return parts[1]
    if len(parts) >= 2 and parts[0] == "d":
        return parts[1]
    return None

def actor_index(action):
    parts = str(action).lower().strip().split()
    if parts and parts[0].startswith("p"):
        try:
            return int(parts[0][1:]) - 1
        except ValueError:
            return None
    return None

def cards_in_action(action):
    return [c.lower() for c in CARD_RE.findall(str(action))]

def hand_action_metrics(hh):
    actions = hand_actions(hh)
    num_actions = len(actions)

    if not actions:
        return {

```

```

        "num_actions": np.nan,
        "num_folds": np.nan,
        "num_calls_checks": np.nan,
        "num_bets_raises": np.nan,
        "num_posts": np.nan,
        "num_showdown_like": np.nan,
        "bet_raise_rate": np.nan,
        "fold_rate": np.nan,
    }

codes = [action_code(a) for a in actions]
num_folds = sum(code == "f" for code in codes)
num_calls_checks = sum(code == "cc" for code in codes)
num_bets_raises = sum(code == "cbr" for code in codes)
num_posts = sum(code in {"pb", "p"} for code in codes)
num_showdown_like = sum(code in {"sm", "sd"} or "show" in a or "muck" in a for

return {
    "num_actions": num_actions,
    "num_folds": num_folds,
    "num_calls_checks": num_calls_checks,
    "num_bets_raises": num_bets_raises,
    "num_posts": num_posts,
    "num_showdown_like": num_showdown_like,
    "bet_raise_rate": num_bets_raises / num_actions if num_actions else np.nan,
    "fold_rate": num_folds / num_actions if num_actions else np.nan,
}

def card_visibility_metrics(hh):
    actions = hand_actions(hh)
    known_cards = []
    unknown_token_count = 0
    show_muck_actions = 0
    known_show_muck_actions = 0

    for action in actions:
        cards = cards_in_action(action)
        known_cards.extend(cards)
        unknown_token_count += action.count("?")

        code = action_code(action)
        is_show_muck = code in {"sm", "sd"} or "show" in action or "muck" in action
        if is_show_muck:
            show_muck_actions += 1
            known_show_muck_actions += int(bool(cards))

    return {
        "has_actions": bool(actions),
        "known_card_count": len(known_cards),
        "unknown_card_token_count": unknown_token_count,
        "has_known_cards": len(known_cards) > 0,
        "has_unknown_cards": unknown_token_count > 0,
        "show_muck_actions": show_muck_actions,
        "known_show_muck_actions": known_show_muck_actions,
        "known_cards": known_cards if known_cards else None,
    }

```

```

}

def stack_net_changes(starting_stacks, finishing_stacks):
    starting = numeric_values(starting_stacks)
    finishing = numeric_values(finishing_stacks)
    if len(starting) == len(finishing) and starting:
        return [finish - start for start, finish in zip(starting, finishing)]
    return []

def parse_card_visibility(path):
    path = Path(path)

    try:
        hh = load_hand(path)
        row = {
            "file": str(path),
            "filename": path.name,
            "parse_success": True,
            "variant": get_attr(hh, "variant", None),
            "year": get_attr(hh, "year", None),
            "num_players": safe_len(get_attr(hh, "players", [])),
            "num_actions": len(hand_actions(hh)),
        }
        row.update(card_visibility_metrics(hh))
        return row

    except Exception as e:
        return {
            "file": str(path),
            "filename": path.name,
            "parse_success": False,
            "parse_error": str(e)[:200],
        }

def parse_winner_outcome(path):
    path = Path(path)

    try:
        hh = load_hand(path)
    except Exception as e:
        return {
            "file": str(path),
            "filename": path.name,
            "parse_success": False,
            "parse_error": str(e)[:200],
        }

    players = to_list(get_attr(hh, "players", []))
    actions = hand_actions(hh)
    num_players = len(players)

    net_changes = stack_net_changes(
        get_attr(hh, "starting_stacks", None),

```

```

        get_attr(hh, "finishing_stacks", None),
    )

    if not net_changes:
        winnings = numeric_values(get_attr(hh, "winnings", None))
        if len(winnings) == num_players:
            net_changes = winnings

    max_net = max(net_changes) if net_changes else None
    winner_indices = [
        i for i, value in enumerate(net_changes)
        if max_net is not None and value == max_net and value > 0
    ]
    winner_names = [players[i] for i in winner_indices if i < len(players)]
    winner_tokens = {f"p{i + 1}" for i in winner_indices}

    known_cards_by_player = {f"p{i + 1}": [] for i in range(num_players)}
    show_muck_by_player = {f"p{i + 1}": 0 for i in range(num_players)}

    for action in actions:
        cards = cards_in_action(action)
        code = action_code(action)
        tokens = set(action.split())

        for token in known_cards_by_player:
            if token in tokens:
                show_muck_by_player[token] += int(code in {"sm", "sd"} or "show" in code)
                if cards:
                    known_cards_by_player[token].extend(cards)

    winner_known_cards = [
        card
        for token in winner_tokens
        for card in known_cards_by_player.get(token, [])
    ]
    winner_show_muck_actions = sum(show_muck_by_player.get(token, 0) for token in winner_tokens)

    action_metrics = hand_action_metrics(hh)
    visibility_metrics = card_visibility_metrics(hh)

    return {
        "file": str(path),
        "filename": path.name,
        "parse_success": True,
        "variant": get_attr(hh, "variant", None),
        "currency": get_attr(hh, "currency", None),
        "year": get_attr(hh, "year", None),
        "players": players,
        "num_players": num_players,
        "starting_stacks": get_attr(hh, "starting_stacks", None),
        "finishing_stacks": get_attr(hh, "finishing_stacks", None),
        "winnings": get_attr(hh, "winnings", None),
        "net_changes": net_changes,
        "winner_indices": winner_indices,
        "winner_names": winner_names,
        "num_winners": len(winner_indices),
    }

```



```

        "winner_known_cards": winner_known_cards if winner_known_cards else None,
        "winner_known_card_count": len(winner_known_cards),
        "winner_has_known_cards": len(winner_known_cards) > 0,
        "winner_show_muck_actions": winner_show_muck_actions,
        "hand_known_card_count": visibility_metrics["known_card_count"],
        "hand_has_known_cards": visibility_metrics["has_known_cards"],
        "unknown_card_token_count": visibility_metrics["unknown_card_token_count"],
        **action_metrics,
        "showdown_like_hand": action_metrics["num_showdown_like"] > 0,
    }

def parse_participant_rows(path):
    path = Path(path)
    hh = load_hand(path)

    players = to_list(get_attr(hh, "players", []))
    actions = hand_actions(hh)
    codes = [action_code(action) for action in actions]
    net_changes = stack_net_changes(
        get_attr(hh, "starting_stacks", None),
        get_attr(hh, "finishing_stacks", None),
    )

    counters = {
        player: {
            "actions": 0,
            "bets_raises": 0,
            "folds": 0,
            "call_checks": 0,
            "show_muck": 0,
        }
        for player in players
    }

    for action, code in zip(actions, codes):
        idx = actor_index(action)
        if idx is None or idx >= len(players):
            continue

        player = players[idx]
        counters[player]["actions"] += 1
        counters[player]["bets_raises"] += int(code == "cbr")
        counters[player]["folds"] += int(code == "f")
        counters[player]["call_checks"] += int(code == "cc")
        counters[player]["show_muck"] += int(code in {"sm", "sd"})

    rows = []
    for i, player in enumerate(players):
        metrics = counters[player]
        net = net_changes[i] if i < len(net_changes) else np.nan
        rows.append({
            "file": str(path),
            "participant": player,
            "seat": i + 1,
            "actions": metrics["actions"],

```

```

        "bets_raises": metrics["bets_raises"],
        "folds": metrics["folds"],
        "call_checks": metrics["call_checks"],
        "show_muck": metrics["show_muck"],
        "active_hand": metrics["actions"] > 0,
        "won_hand": net > 0 if pd.notna(net) else np.nan,
        "net_change": net,
    })

    return rows

def parse_obfuscation_resolution(path):
    path = Path(path)

    try:
        hh = load_hand(path)
    except Exception as e:
        return {
            "file": str(path),
            "filename": path.name,
            "parse_success": False,
            "parse_error": str(e)[:200],
        }

    actions = hand_actions(hh)

    deal_down_actions = 0
    deal_down_known_actions = 0
    deal_down_obfuscated_actions = 0
    sm_actions = 0
    sm_known_actions = 0
    sm_obfuscated_actions = 0
    sm_no_card_actions = 0
    last_non_deal_action = None
    last_non_deal_code = None

    known_card_count = 0
    unknown_char_count = 0

    for action in actions:
        code = action_code(action)
        cards = cards_in_action(action)
        has_known = bool(cards)
        has_unknown = "?" in action

        known_card_count += len(cards)
        unknown_char_count += action.count("?")

        if code == "dh":
            deal_down_actions += 1
            deal_down_known_actions += int(has_known)
            deal_down_obfuscated_actions += int(has_unknown)

        if code == "sm":
            sm_actions += 1

```

```

        sm_known_actions += int(has_known)
        sm_obfuscated_actions += int(has_unknown and not has_known)
        sm_no_card_actions += int(not has_known and not has_unknown)

    if code not in {"dh", "db"} and code is not None:
        last_non_deal_action = action
        last_non_deal_code = code

if sm_actions == 0:
    resolution_type = "No show/muck activity"
elif sm_known_actions > 0:
    resolution_type = "Show/muck with known cards"
elif sm_obfuscated_actions > 0:
    resolution_type = "Show/muck with obfuscated cards"
else:
    resolution_type = "Show/muck with no card text"

last_action = actions[-1] if actions else None

return {
    "file": str(path),
    "filename": path.name,
    "parse_success": True,
    "variant": get_attr(hh, "variant", None),
    "num_players": safe_len(get_attr(hh, "players", [])),
    "num_actions": len(actions),
    "known_card_count": known_card_count,
    "unknown_char_count": unknown_char_count,
    "has_known_cards": known_card_count > 0,
    "has_any_obfuscation": unknown_char_count > 0,
    "deal_down_actions": deal_down_actions,
    "deal_down_known_actions": deal_down_known_actions,
    "deal_down_obfuscated_actions": deal_down_obfuscated_actions,
    "sm_actions": sm_actions,
    "sm_known_actions": sm_known_actions,
    "sm_obfuscated_actions": sm_obfuscated_actions,
    "sm_no_card_actions": sm_no_card_actions,
    "has_sm": sm_actions > 0,
    "resolution_type": resolution_type,
    "last_action": last_action,
    "last_action_code": action_code(last_action) if last_action else None,
    "last_non_deal_action": last_non_deal_action,
    "last_non_deal_code": last_non_deal_code,
}

```

## Parse Single-Hand PHH Files

The main analysis uses the single-hand `.phh` files as the unit of observation. Each parsed row represents one hand-level event record with metadata, participant counts, action counts, stack information, and parse diagnostics.

```
In [3]: def parse_phh_with_pokerkit(path):
        path = Path(path)
```

```

try:
    with open(path, "rb") as f:
        hh = HandHistory.load(f)

    parse_success = True
    parse_error = None

except Exception as e:
    return {
        "file": str(path),
        "filename": path.name,
        "parse_success": False,
        "parse_error": str(e)[:300],
        **parse_path_metadata(path),
    }

players = get_attr(hh, "players", None)
blinds = get_attr(hh, "blinds_or_straddles", None)
antes = get_attr(hh, "antes", None)
starting_stacks = get_attr(hh, "starting_stacks", None)

small_blind = np.nan
big_blind = np.nan

if isinstance(blinds, (list, tuple)):
    numeric_blinds = [b for b in blinds if isinstance(b, (int, float))]
    if len(numeric_blinds) >= 1:
        small_blind = numeric_blinds[0]
    if len(numeric_blinds) >= 2:
        big_blind = numeric_blinds[1]

total_starting_stack = safe_sum_numeric(starting_stacks)

avg_starting_stack = (
    total_starting_stack / len(starting_stacks)
    if isinstance(starting_stacks, (list, tuple)) and len(starting_stacks) > 0
    else np.nan
)

row = {
    "file": str(path),
    "filename": path.name,
    "parse_success": parse_success,
    "parse_error": parse_error,

    "variant": get_attr(hh, "variant", None),
    "currency": get_attr(hh, "currency", None),
    "year": get_attr(hh, "year", None),
    "city": get_attr(hh, "city", None),
    "country": get_attr(hh, "country", None),

    "players": players,
    "num_players": safe_len(players),

    "blinds_or_straddles": blinds,

```



|   | file                                              | filename                  | parse_success | parse_error | variant | currency |
|---|---------------------------------------------------|---------------------------|---------------|-------------|---------|----------|
| 0 | C:\Users\J1999\Downloads\poker-hand-histories\... | alice-carol-wikipedia.phh | True          | None        | FB      | None     |
| 1 | C:\Users\J1999\Downloads\poker-hand-histories\... | antonius-blom-2009.phh    | True          | None        | PO      | USD      |
| 2 | C:\Users\J1999\Downloads\poker-hand-histories\... | arieh-yockey-2019.phh     | True          | None        | F2L3D   | None     |
| 3 | C:\Users\J1999\Downloads\poker-hand-histories\... | dwan-ivey-2009.phh        | True          | None        | NT      | USD      |
| 4 | C:\Users\J1999\Downloads\poker-hand-histories\... | phua-xuan-2019.phh        | True          | None        | NS      | GB       |

5 rows × 31 columns

## Metric Dictionary

These metrics are intentionally business-facing:

- **Actions per hand:** engagement depth; more actions usually mean a longer interaction sequence.
- **Bets/raises per hand:** count of higher-commitment actions.
- **Bet/raise rate:** share of total actions that represent higher-commitment behavior.
- **Showdown-like hand:** a hand with show/muck-style terminal activity; this is a visibility/resolution proxy, not proof of complete card visibility.
- **Winner net gain:** observed positive outcome size where stack/winnings data can be parsed; useful for outcome intensity, not player skill.

```
In [4]: import matplotlib.pyplot as plt
import matplotlib.ticker as mtick
import numpy as np
```

```
BG = "#f7f6f2"
TEXT = "#28251d"
GRID = "#dcd9d5"
TEAL = "#01696f"
```

```

RED = "#EF553B"
GRAY = "#bab9b4"
GREEN = "#00A676"

plt.rcParams.update({
    "figure.facecolor": BG,
    "axes.facecolor": BG,
    "axes.edgecolor": TEXT,
    "axes.labelcolor": TEXT,
    "xtick.color": TEXT,
    "ytick.color": TEXT,
    "text.color": TEXT,
    "font.size": 10,
})

```

```

In [5]: analysis = phh_flat.copy()

analysis["source_type"] = np.where(analysis["is_wsop"], "WSOP", "Other / Example Ha
analysis["variant_clean"] = analysis["variant"].fillna("Unknown").astype(str)

def player_count_bucket(n):
    if pd.isna(n):
        return "Unknown"
    elif n <= 2:
        return "Heads-Up / 2-Handed"
    elif n <= 6:
        return "Short-Handed (3-6)"
    else:
        return "Full-Ring (7+)"

analysis["player_count_bucket"] = analysis["num_players"].apply(player_count_bucket

analysis["actions_per_player"] = analysis["num_actions"] / analysis["num_players"]
analysis["bets_raises_per_player"] = analysis["num_bets_raises"] / analysis["num_pl
analysis["showdown_like_hand"] = analysis["num_showdown_like"].fillna(0) > 0

analysis.head()

```

Out[5]:

|   |                                                   | file                      | filename | parse_success | parse_error | variant | curren |
|---|---------------------------------------------------|---------------------------|----------|---------------|-------------|---------|--------|
| 0 | C:\Users\J1999\Downloads\poker-hand-histories\... | alice-carol-wikipedia.phh |          | True          | None        | FB      | Nc     |
| 1 | C:\Users\J1999\Downloads\poker-hand-histories\... | antonius-blom-2009.phh    |          | True          | None        | PO      | U      |
| 2 | C:\Users\J1999\Downloads\poker-hand-histories\... | arieh-yockey-2019.phh     |          | True          | None        | F2L3D   | Nc     |
| 3 | C:\Users\J1999\Downloads\poker-hand-histories\... | dwan-ivey-2009.phh        |          | True          | None        | NT      | U      |
| 4 | C:\Users\J1999\Downloads\poker-hand-histories\... | phua-xuan-2019.phh        |          | True          | None        | NS      | G      |

5 rows × 37 columns

```
In [6]: commercial_summary = pd.DataFrame([
    "hands_analyzed": len(analysis),
    "parse_success_rate": analysis["parse_success"].mean(),
    "state_create_success_rate": analysis["state_create_success"].mean(),
    "avg_players_per_hand": analysis["num_players"].mean(),
    "median_players_per_hand": analysis["num_players"].median(),
    "avg_actions_per_hand": analysis["num_actions"].mean(),
    "avg_bets_raises_per_hand": analysis["num_bets_raises"].mean(),
    "avg_bet_raise_rate": analysis["bet_raise_rate"].mean(),
    "showdown_like_rate": analysis["showdown_like_hand"].mean(),
    ])

commercial_summary.T
```



Out[6]:

0

|                                  |              |
|----------------------------------|--------------|
| <b>hands_analyzed</b>            | 10088.000000 |
| <b>parse_success_rate</b>        | 1.000000     |
| <b>state_create_success_rate</b> | 1.000000     |
| <b>avg_players_per_hand</b>      | 5.987807     |
| <b>median_players_per_hand</b>   | 6.000000     |
| <b>avg_actions_per_hand</b>      | 16.688243    |
| <b>avg_bets_raises_per_hand</b>  | 1.857950     |
| <b>avg_bet_raise_rate</b>        | 0.105152     |
| <b>showdown_like_rate</b>        | 0.169905     |

## Engagement Depth

The first commercial question is how much interaction the product creates. Action count is a simple but useful proxy for participant engagement because it measures how many observable events occur before a hand resolves.

```
In [7]: action_df = analysis.dropna(subset=["num_actions"]).copy()

fig, ax = plt.subplots(figsize=(12, 6))

ax.hist(
    action_df["num_actions"],
    bins=30,
    color=TEAL,
    edgecolor=TEXT,
    alpha=0.85,
    zorder=3
)

ax.axvline(
    action_df["num_actions"].median(),
    color=RED,
    linestyle="--",
    linewidth=2,
    label=f"Median: {action_df['num_actions'].median():.0f} actions"
)

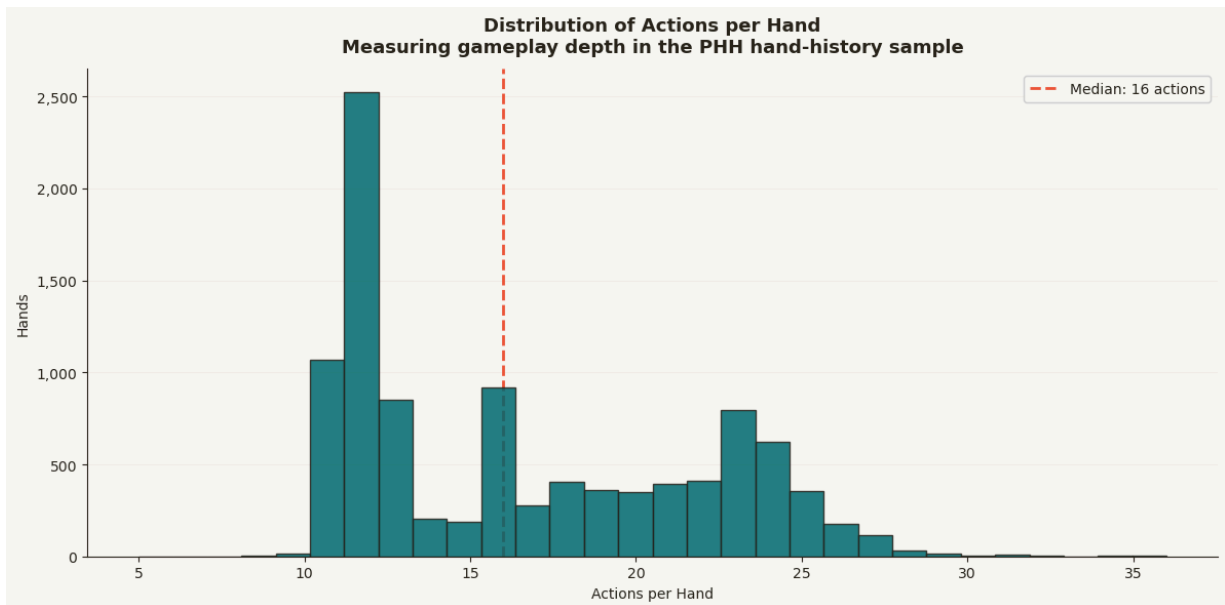
ax.set_title(
    "Distribution of Actions per Hand\n"
    "Measuring gameplay depth in the PHH hand-history sample",
    fontsize=13,
    fontweight="bold",
    pad=12
)
```

```

ax.set_xlabel("Actions per Hand")
ax.set_ylabel("Hands")
ax.yaxis.set_major_formatter(mtick.StrMethodFormatter("{x:,.0f}"))
ax.grid(axis="y", alpha=0.25, color=GRID, zorder=0)
ax.legend(framealpha=0.9)
ax.spines[["top", "right"]].set_visible(False)

plt.tight_layout()
plt.savefig("phh_actions_per_hand_distribution.png", dpi=150, bbox_inches="tight",
plt.show()

```



## Interpretation: Engagement Depth

This distribution is more useful than a poker-specific hand review. It tells a product or commercial analytics audience how frequently the experience is quick and shallow versus longer and more involved.

**\*\*Commercial read:\*\*** action depth can become a monitoring KPI for game quality, player experience, and contest pacing.

## Higher-Commitment Action Intensity

Bet/raise share is used as a lightweight proxy for interaction intensity. It should not be interpreted as a complete poker-theory aggression metric; it simply measures how often observable actions involve higher commitment.

```

In [8]: bet_df = analysis.dropna(subset=["bet_raise_rate"]).copy()

fig, ax = plt.subplots(figsize=(12, 6))

ax.hist(

```

```

bet_df["bet_raise_rate"] * 100,
bins=25,
color=RED,
edgecolor=TEXT,
alpha=0.85,
zorder=3
)

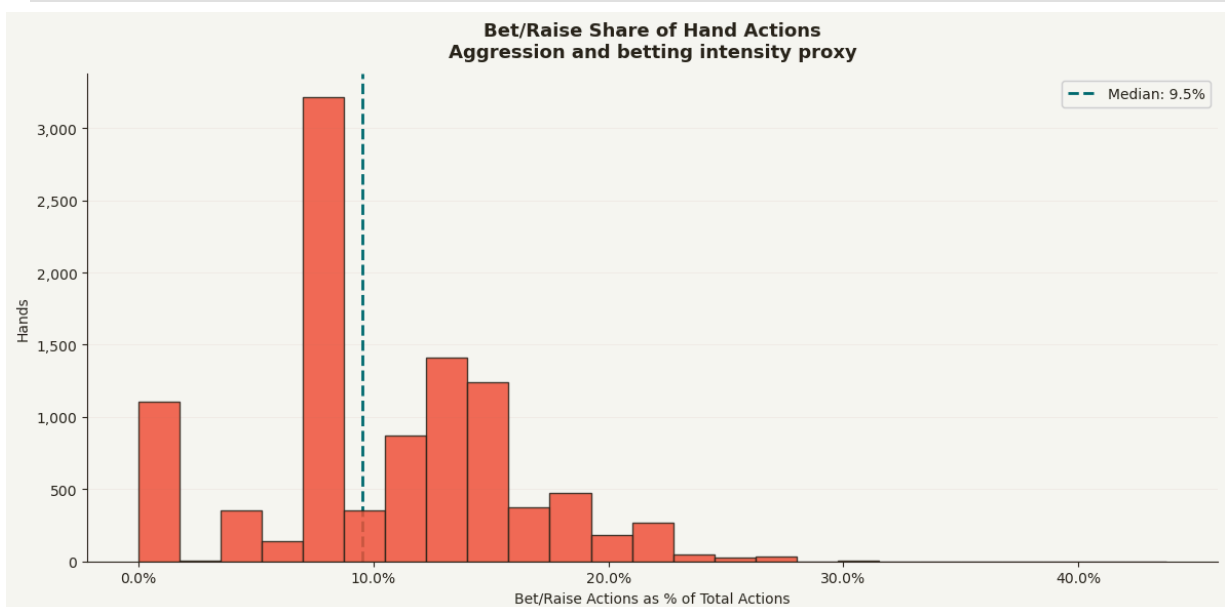
ax.axvline(
    bet_df["bet_raise_rate"].median() * 100,
    color=TEAL,
    linestyle="--",
    linewidth=2,
    label=f"Median: {bet_df['bet_raise_rate'].median() * 100:.1f}%"
)

ax.set_title(
    "Bet/Raise Share of Hand Actions\n"
    "Aggression and betting intensity proxy",
    fontsize=13,
    fontweight="bold",
    pad=12
)

ax.set_xlabel("Bet/Raise Actions as % of Total Actions")
ax.set_ylabel("Hands")
ax.xaxis.set_major_formatter(mtick.PercentFormatter())
ax.yaxis.set_major_formatter(mtick.StrMethodFormatter("{x:,.0f}"))
ax.grid(axis="y", alpha=0.25, color=GRID, zorder=0)
ax.legend(framealpha=0.9)
ax.spines[["top", "right"]].set_visible(False)

plt.tight_layout()
plt.savefig("phh_bet_raise_rate_distribution.png", dpi=150, bbox_inches="tight", fa
plt.show()

```



**Interpretation: Betting Intensity**

Higher betting intensity can make a format feel more action-oriented, while lower intensity may indicate a more passive or approachable experience. For a gaming operator, this type of metric can support product positioning, game monitoring, and participant-experience comparisons.

## Action-Based Experience Segments

Rather than treating every hand as interchangeable inventory, the notebook groups hands into low-, medium-, and high-action segments. This converts raw hand histories into a product analytics segmentation.

```
In [9]: seg_df = analysis.dropna(subset=["num_actions", "bet_raise_rate"]).copy()

seg_df["action_segment"] = pd.qcut(
    seg_df["num_actions"],
    q=3,
    labels=["Low-Action Hands", "Medium-Action Hands", "High-Action Hands"]
)

segment_summary = (
    seg_df
    .groupby("action_segment", as_index=False, observed=True)
    .agg(
        hands=("file", "count"),
        avg_actions=("num_actions", "mean"),
        avg_bets_raises=("num_bets_raises", "mean"),
        avg_bet_raise_rate=("bet_raise_rate", "mean"),
        showdown_like_rate=("showdown_like_hand", "mean"),
        avg_players=("num_players", "mean")
    )
)

segment_summary["hand_share"] = segment_summary["hands"] / segment_summary["hands"]

segment_summary
```

```
Out[9]:
```

|   | action_segment      | hands | avg_actions | avg_bets_raises | avg_bet_raise_rate | showdown_like_ |
|---|---------------------|-------|-------------|-----------------|--------------------|----------------|
| 0 | Low-Action Hands    | 3606  | 11.693012   | 0.707432        | 0.059075           | 0.000          |
| 1 | Medium-Action Hands | 3202  | 15.691443   | 2.182074        | 0.139997           | 0.003          |
| 2 | High-Action Hands   | 3280  | 23.153049   | 2.806402        | 0.121793           | 0.515          |

```
In [10]: fig, ax = plt.subplots(figsize=(12, 6))

x = np.arange(len(segment_summary))
```

```

width = 0.36

ax.bar(
    x - width / 2,
    segment_summary["avg_bets_raises"],
    width,
    label="Avg Bets/Raises",
    color=RED,
    zorder=3
)

ax.bar(
    x + width / 2,
    segment_summary["showdown_like_rate"] * 100,
    width,
    label="Showdown-Like Rate (%)",
    color=TEAL,
    zorder=3
)

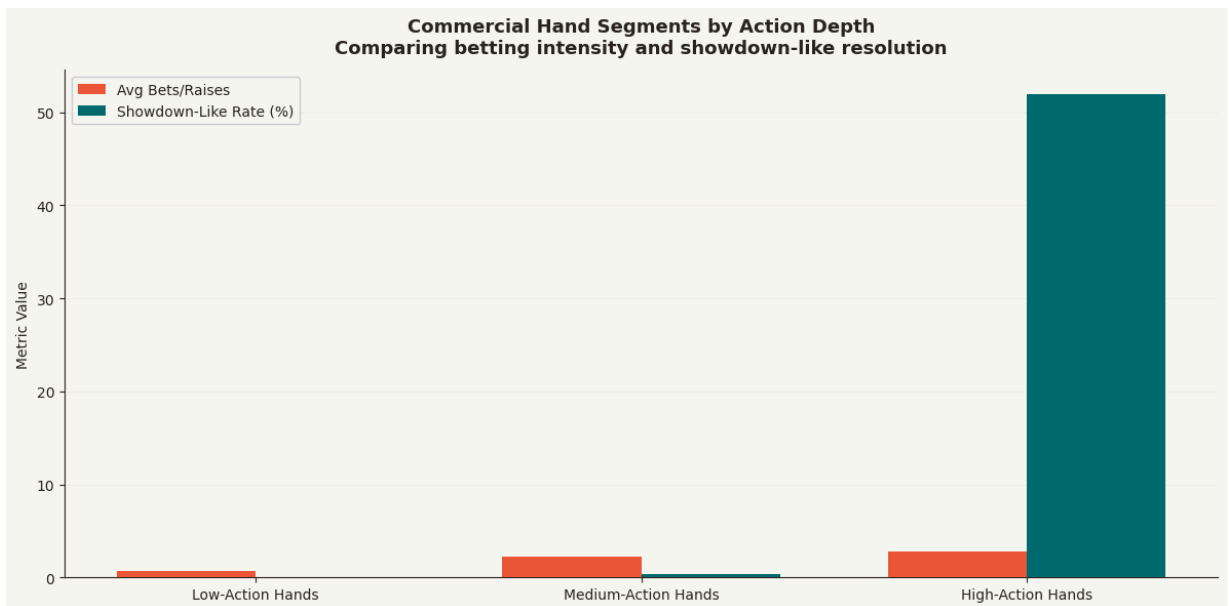
ax.set_xticks(x)
ax.set_xticklabels(segment_summary["action_segment"])

ax.set_title(
    "Commercial Hand Segments by Action Depth\n"
    "Comparing betting intensity and showdown-like resolution",
    fontsize=13,
    fontweight="bold",
    pad=12
)

ax.set_xlabel("")
ax.set_ylabel("Metric Value")
ax.grid(axis="y", alpha=0.25, color=GRID, zorder=0)
ax.legend(framealpha=0.9)
ax.spines[["top", "right"]].set_visible(False)

plt.tight_layout()
plt.savefig("phh_hand_segments_by_action_depth.png", dpi=150, bbox_inches="tight",
plt.show()

```



```
In [11]: segment_display = segment_summary.copy()

segment_display["hand_share"] = segment_display["hand_share"].map(lambda x: f"{x:.1}")
segment_display["avg_actions"] = segment_display["avg_actions"].round(2)
segment_display["avg_bets_raises"] = segment_display["avg_bets_raises"].round(2)
segment_display["avg_bet_raise_rate"] = segment_display["avg_bet_raise_rate"].map(lambda x: f"{x:.1}%")
segment_display["showdown_like_rate"] = segment_display["showdown_like_rate"].map(lambda x: f"{x:.1}%")
segment_display["avg_players"] = segment_display["avg_players"].round(2)

segment_display.rename(columns={
    "action_segment": "Hand Segment",
    "hands": "Hands",
    "hand_share": "Hand Share",
    "avg_actions": "Avg Actions",
    "avg_bets_raises": "Avg Bets/Raises",
    "avg_bet_raise_rate": "Avg Bet/Raise Rate",
    "showdown_like_rate": "Showdown-Like Rate",
    "avg_players": "Avg Players"
})
```

Out[11]:

|   | Hand Segment        | Hands | Avg Actions | Avg Bets/Raises | Avg Bet/Raise Rate | Showdown-Like Rate | Avg Players | Hand Share |
|---|---------------------|-------|-------------|-----------------|--------------------|--------------------|-------------|------------|
| 0 | Low-Action Hands    | 3606  | 11.69       | 0.71            | 5.9%               | 0.0%               | 5.99        | 35.7%      |
| 1 | Medium-Action Hands | 3202  | 15.69       | 2.18            | 14.0%              | 0.3%               | 5.99        | 31.7%      |
| 2 | High-Action Hands   | 3280  | 23.15       | 2.81            | 12.2%              | 52.0%              | 5.98        | 32.5%      |

## Interpretation: Experience Segments

The high-action segment represents hands with deeper interaction sequences and more observable participant engagement. The low-action segment is not necessarily bad; it may represent faster, simpler, or less intimidating gameplay.

**\*\*Commercial read:\*\*** this segmentation could be tracked across game formats, stakes, player cohorts, time periods, or acquisition channels in a production analytics environment.

## Contest Lifecycle Depth

Action count measures how many observable events occurred, but a lifecycle view is easier for non-poker stakeholders to understand. This section groups hands by how far they progressed through the public contest sequence before resolving.

The business question is: **\*\*what share of interactions end quickly versus becoming deeper, more visible experiences?\*\***

```
In [12]: def classify_lifecycle_depth(path):
    path = Path(path)
    with open(path, "rb") as f:
        hh = HandHistory.load(f)

    actions = [str(a).lower().strip() for a in getattr(hh, "actions", [])]
    codes = [action_code(a) for a in actions]

    board_deals = sum(code == "db" for code in codes)
    if board_deals == 0:
        lifecycle_stage = "No Board Reveal"
    elif board_deals == 1:
        lifecycle_stage = "First Board Reveal"
    elif board_deals == 2:
        lifecycle_stage = "Second Board Reveal"
    else:
        lifecycle_stage = "Final Board Reveal"

    first_board_index = next((i for i, code in enumerate(codes) if code == "db"), None)
    pre_board_actions = sum(
        code in {"f", "cc", "cbr"}
        for i, code in enumerate(codes)
        if first_board_index is None or i < first_board_index
    )
    post_board_actions = sum(
        code in {"f", "cc", "cbr"}
        for i, code in enumerate(codes)
        if first_board_index is not None and i > first_board_index
    )

    last_non_deal_code = None
```





| Out[12]: | lifecycle_stage        | hands | avg_actions | avg_pre_board_actions | avg_post_board_actions | avg_ |
|----------|------------------------|-------|-------------|-----------------------|------------------------|------|
| <b>2</b> | No Board<br>Reveal     | 4720  | 12.139619   | 6.096610              | 0.000000               |      |
| <b>1</b> | First Board<br>Reveal  | 1488  | 16.302419   | 6.265457              | 3.039651               |      |
| <b>3</b> | Second Board<br>Reveal | 1109  | 19.438233   | 6.208296              | 5.232642               |      |
| <b>0</b> | Final Board<br>Reveal  | 2771  | 23.542764   | 6.270660              | 7.042584               |      |

```

In [13]: fig, ax = plt.subplots(figsize=(12, 6))

bars = ax.bar(
    lifecycle_summary["lifecycle_stage"].astype(str),
    lifecycle_summary["hand_share"] * 100,
    color=[GRAY, TEAL, TEAL, RED],
    zorder=3,
)

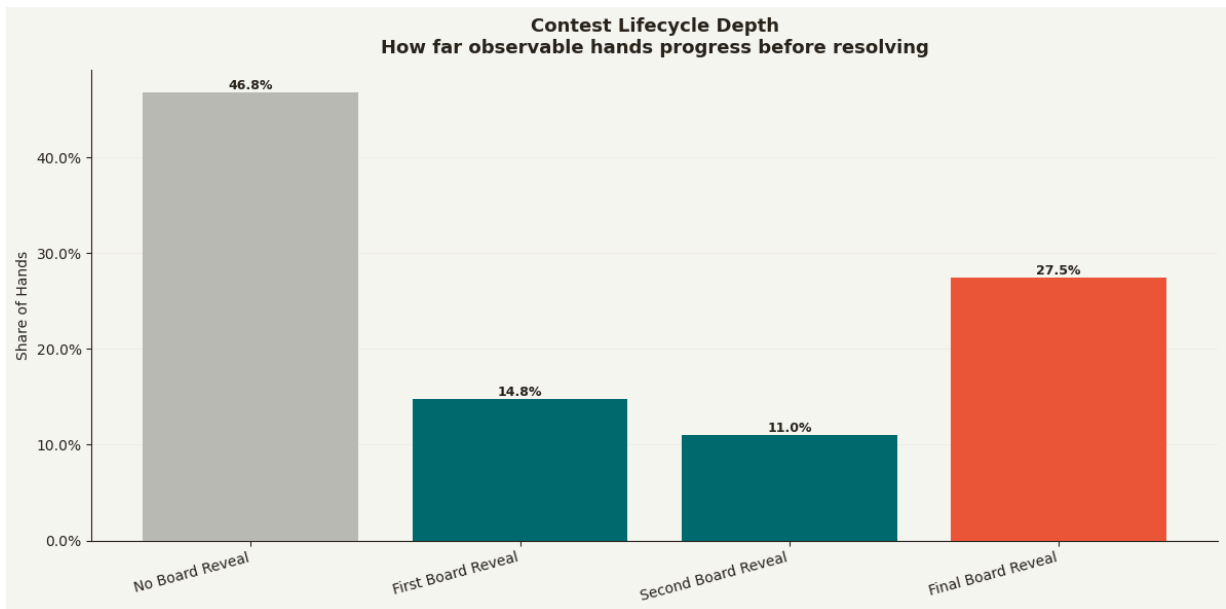
for bar in bars:
    height = bar.get_height()
    ax.text(
        bar.get_x() + bar.get_width() / 2,
        height,
        f"{height:.1f}%",
        ha="center",
        va="bottom",
        fontsize=9,
        fontweight="bold",
    )

ax.set_title(
    "Contest Lifecycle Depth\n"
    "How far observable hands progress before resolving",
    fontsize=13,
    fontweight="bold",
    pad=12,
)

ax.set_xlabel("")
ax.set_ylabel("Share of Hands")
ax.yaxis.set_major_formatter(mtick.PercentFormatter())
ax.grid(axis="y", alpha=0.25, color=GRID, zorder=0)
ax.spines[["top", "right"]].set_visible(False)

plt.xticks(rotation=15, ha="right")
plt.tight_layout()
plt.savefig("phh_contest_lifecycle_depth.png", dpi=150, bbox_inches="tight", faceco
plt.show()

```



## Interpretation: Contest Lifecycle Depth

Lifecycle depth translates hand histories into a product funnel. Hands that resolve before any board reveal are quick, low-depth interactions. Hands that reach the final board reveal are deeper experiences with more observable decision points and clearer resolution moments.

**\*\*Commercial read:\*\*** a gaming operator could track lifecycle depth to understand contest pacing, compare formats, and monitor whether the product is producing the intended mix of quick interactions and deeper engagement moments.

## Card Visibility and Data Quality

This is the most important caveat in the project. The PHH logs do not consistently expose private cards or winner-side cards. That limits strategy and hand-strength analysis, but it does not eliminate the value of action-based behavioral analytics.

```
In [14]: card_rows = [parse_card_visibility(path) for path in tqdm(phh_files, desc="Scanning  
card_df = pd.DataFrame(card_rows)  
  
card_df.head()
```

```
Scanning card visibility: 100%|████████████████████████████████████████████████████████████████████████████████| 100  
88/10088 [00:07<00:00, 1425.59it/s]
```

|   | file                                              | filename                  | parse_success | variant | year   | num_playe |
|---|---------------------------------------------------|---------------------------|---------------|---------|--------|-----------|
| 0 | C:\Users\J1999\Downloads\poker-hand-histories\... | alice-carol-wikipedia.phh | True          | FB      | NaN    |           |
| 1 | C:\Users\J1999\Downloads\poker-hand-histories\... | antonius-blom-2009.phh    | True          | PO      | 2009.0 |           |
| 2 | C:\Users\J1999\Downloads\poker-hand-histories\... | arieh-yockey-2019.phh     | True          | F2L3D   | 2019.0 |           |
| 3 | C:\Users\J1999\Downloads\poker-hand-histories\... | dwan-ivey-2009.phh        | True          | NT      | 2009.0 |           |
| 4 | C:\Users\J1999\Downloads\poker-hand-histories\... | phua-xuan-2019.phh        | True          | NS      | 2019.0 |           |

```
In [15]: card_summary = pd.DataFrame([
    "hands": len(card_df),
    "parse_success_rate": card_df["parse_success"].mean(),
    "hands_with_known_cards": card_df["has_known_cards"].mean(),
    "hands_with_unknown_cards": card_df["has_unknown_cards"].mean(),
    "avg_known_card_count": card_df["known_card_count"].mean(),
    "median_known_card_count": card_df["known_card_count"].median(),
    "avg_show_muck_actions": card_df["show_muck_actions"].mean(),
    "avg_known_show_muck_actions": card_df["known_show_muck_actions"].mean(),
])

card_summary.T
```

|                                    | 0            |
|------------------------------------|--------------|
| <b>hands</b>                       | 10088.000000 |
| <b>parse_success_rate</b>          | 1.000000     |
| <b>hands_with_known_cards</b>      | 0.387589     |
| <b>hands_with_unknown_cards</b>    | 0.001883     |
| <b>avg_known_card_count</b>        | 0.675555     |
| <b>median_known_card_count</b>     | 0.000000     |
| <b>avg_show_muck_actions</b>       | 0.349921     |
| <b>avg_known_show_muck_actions</b> | 0.001487     |

```
In [16]: analysis_cards = analysis.merge(
    card_df[[
        "file",
        "known_card_count",
        "unknown_card_token_count",
        "has_known_cards",
        "has_unknown_cards",
```

```

        "show_muck_actions",
        "known_show_muck_actions",
        "known_cards"
    ]],
    on="file",
    how="left"
)

analysis_cards.head()

```

Out[16]:

|   | file                                              | filename                  | parse_success | parse_error | variant | curren |
|---|---------------------------------------------------|---------------------------|---------------|-------------|---------|--------|
| 0 | C:\Users\J1999\Downloads\poker-hand-histories\... | alice-carol-wikipedia.phh | True          | None        | FB      | Nc     |
| 1 | C:\Users\J1999\Downloads\poker-hand-histories\... | antonius-blom-2009.phh    | True          | None        | PO      | U      |
| 2 | C:\Users\J1999\Downloads\poker-hand-histories\... | arieh-yockey-2019.phh     | True          | None        | F2L3D   | Nc     |
| 3 | C:\Users\J1999\Downloads\poker-hand-histories\... | dwan-ivey-2009.phh        | True          | None        | NT      | U      |
| 4 | C:\Users\J1999\Downloads\poker-hand-histories\... | phua-xuan-2019.phh        | True          | None        | NS      | G      |

5 rows × 44 columns

In [17]:

```

import matplotlib.pyplot as plt
import matplotlib.ticker as mtick

plot_df = analysis_cards.dropna(subset=["num_actions", "has_known_cards"]).copy()

known_summary = (
    plot_df
    .groupby("has_known_cards", as_index=False)
    .agg(
        hands=("file", "count"),
        avg_actions=("num_actions", "mean"),
        avg_bets_raises=("num_bets_raises", "mean"),
        avg_bet_raise_rate=("bet_raise_rate", "mean"),
        showdown_like_rate=("showdown_like_hand", "mean")
    )
)

known_summary["label"] = np.where(

```

```

        known_summary["has_known_cards"],
        "Known Cards Revealed",
        "No Known Cards Revealed"
    )

fig, ax = plt.subplots(figsize=(10, 6))

x = np.arange(len(known_summary))
width = 0.36

ax.bar(
    x - width / 2,
    known_summary["avg_actions"],
    width,
    label="Avg Actions",
    color=TEAL,
    zorder=3
)

ax.bar(
    x + width / 2,
    known_summary["avg_bets_raises"],
    width,
    label="Avg Bets/Raises",
    color=RED,
    zorder=3
)

ax.set_xticks(x)
ax.set_xticklabels(known_summary["label"])

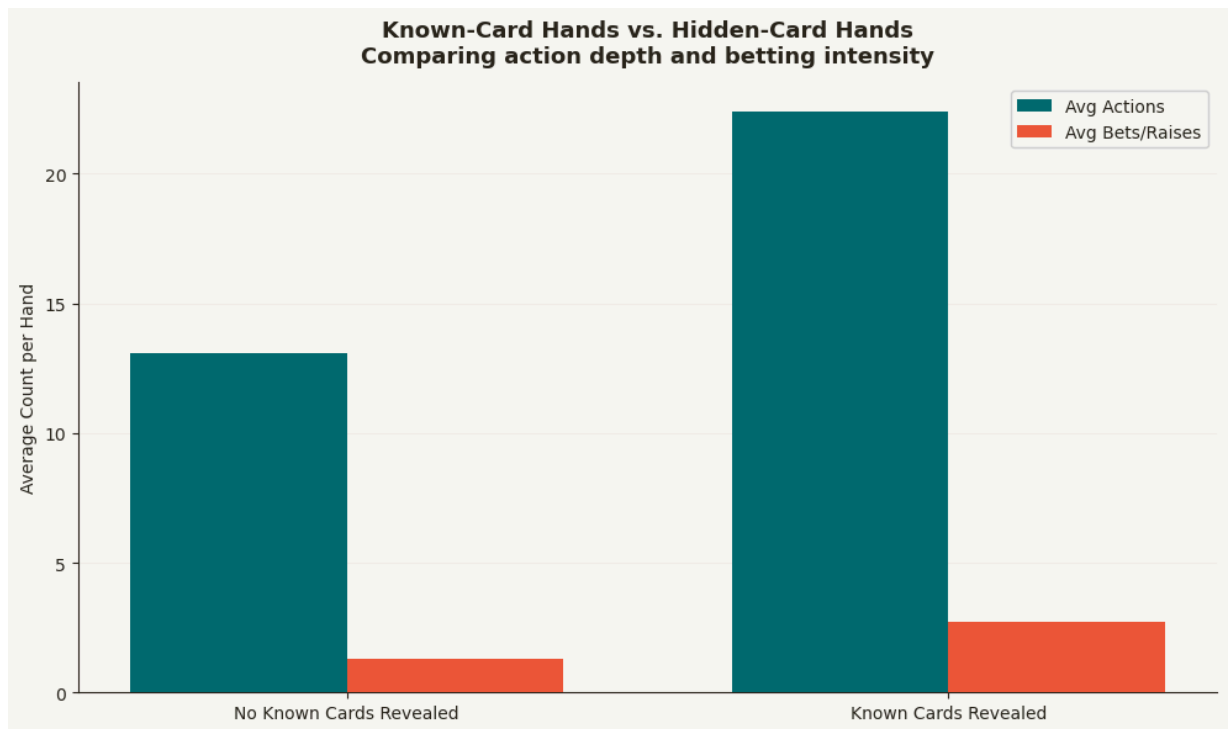
ax.set_title(
    "Known-Card Hands vs. Hidden-Card Hands\n"
    "Comparing action depth and betting intensity",
    fontsize=13,
    fontweight="bold",
    pad=12
)

ax.set_ylabel("Average Count per Hand")
ax.grid(axis="y", alpha=0.25, color=GRID, zorder=0)
ax.legend(framealpha=0.9)
ax.spines[["top", "right"]].set_visible(False)

plt.tight_layout()
plt.savefig("phh_known_cards_vs_hidden_cards.png", dpi=150, bbox_inches="tight", fa
plt.show()

known_summary

```



Out[17]:

|   | has_known_cards | hands | avg_actions | avg_bets_raises | avg_bet_raise_rate | showdown_like |
|---|-----------------|-------|-------------|-----------------|--------------------|---------------|
| 0 | False           | 6178  | 13.080123   | 1.318388        | 0.094788           | 0.00          |
| 1 | True            | 3910  | 22.389258   | 2.710486        | 0.121529           | 0.43          |

```
In [18]: visibility_counts = pd.DataFrame({
    "category": [
        "All Hands",
        "Hands with Unknown/Obfuscated Cards",
        "Hands with Known Cards",
        "Hands with Known Show/Muck Cards"
    ],
    "hands": [
        len(card_df),
        int(card_df["has_unknown_cards"].sum()),
        int(card_df["has_known_cards"].sum()),
        int((card_df["known_show_muck_actions"].fillna(0) > 0).sum())
    ]
})

fig, ax = plt.subplots(figsize=(11, 6))

bars = ax.bar(
    visibility_counts["category"],
    visibility_counts["hands"],
    color=[GRAY, GRAY, TEAL, RED],
    zorder=3
```

```

)

for bar in bars:
    height = bar.get_height()
    ax.text(
        bar.get_x() + bar.get_width() / 2,
        height,
        f"{height:,.0f}",
        ha="center",
        va="bottom",
        fontsize=9,
        fontweight="bold"
    )

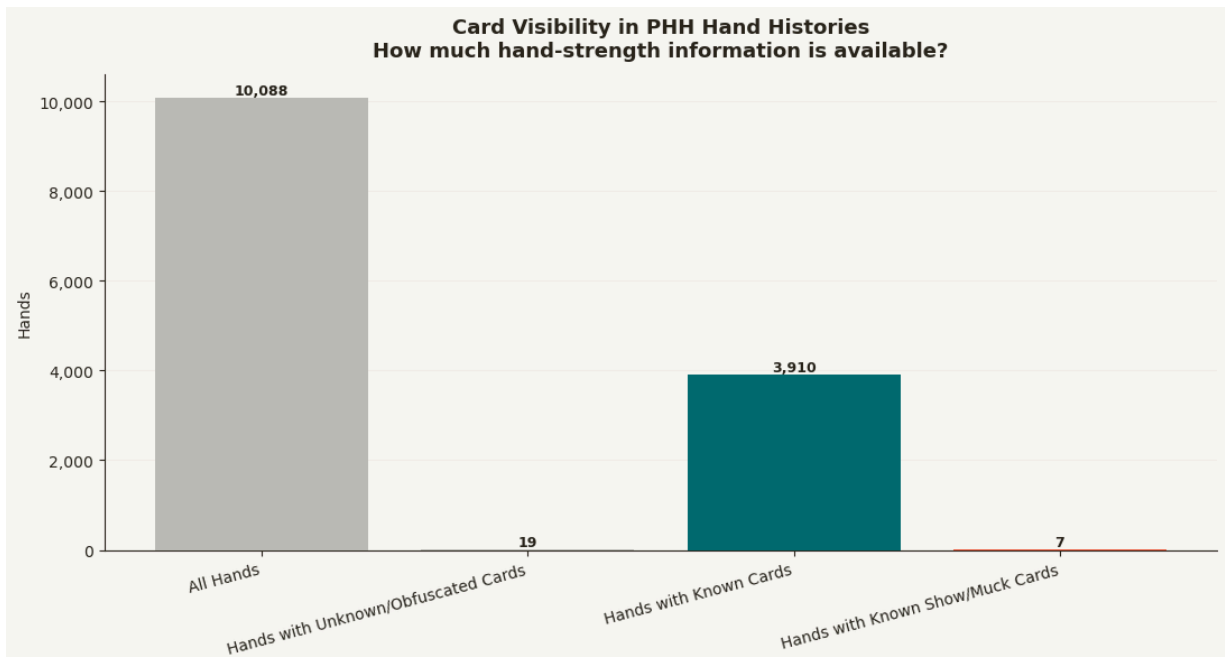
ax.set_title(
    "Card Visibility in PHH Hand Histories\n"
    "How much hand-strength information is available?",
    fontsize=13,
    fontweight="bold",
    pad=12
)

ax.set_ylabel("Hands")
ax.set_xticks(range(len(visibility_counts)))
ax.set_xticklabels(visibility_counts["category"], rotation=15, ha="right")
ax.yaxis.set_major_formatter(mtick.StrMethodFormatter("{x:,.0f}"))
ax.grid(axis="y", alpha=0.25, color=GRID, zorder=0)
ax.spines[["top", "right"]].set_visible(False)

plt.tight_layout()
plt.savefig("phh_card_visibility_funnel.png", dpi=150, bbox_inches="tight", facecol
plt.show()

visibility_counts

```



Out[18]:

|   | category                            | hands |
|---|-------------------------------------|-------|
| 0 | All Hands                           | 10088 |
| 1 | Hands with Unknown/Obfuscated Cards | 19    |
| 2 | Hands with Known Cards              | 3910  |
| 3 | Hands with Known Show/Muck Cards    | 7     |

## Interpretation: Visibility Constraints

Known-card availability is limited and uneven. The dataset should therefore be treated as an event-log dataset, not a complete decision-quality dataset.

**\*\*Commercial read:\*\*** visibility itself is a data-quality metric. An operator with complete internal logs could extend this workflow to connect engagement depth, outcome size, and player behavior to richer card and account-level features.

## Resolution Path Breakdown

The terminal action gives a simple view of how hands resolve. This is not a card-visibility measure; it is a resolution-path measure.

```
In [19]: resolution_map = {
    "f": "Resolved Before Show/Muck",
    "sm": "Show/Muck-Like Resolution",
    "sd": "Showdown-Like Resolution",
}

resolution_summary = (
    lifecycle_analysis
    .assign(resolution_path=lambda df: df["terminal_code"].map(resolution_map).fill
    .groupby("resolution_path", as_index=False)
    .agg(
        hands=("file", "count"),
        avg_actions=("num_actions", "mean"),
        avg_bets_raises=("num_bets_raises", "mean"),
        avg_lifecycle_board_deals=("board_deals", "mean"),
    )
    .sort_values("hands", ascending=False)
)

resolution_summary["hand_share"] = resolution_summary["hands"] / resolution_summary
resolution_summary
```



| Out[19]: | resolution_path           | hands | avg_actions | avg_bets_raises | avg_lifecycle_board_deals | hand_sha |
|----------|---------------------------|-------|-------------|-----------------|---------------------------|----------|
| <b>0</b> | Resolved Before Show/Muck | 8374  | 15.152615   | 1.720325        | 0.828039                  | 0.8300   |
| <b>1</b> | Show/Muck-Like Resolution | 1714  | 24.190782   | 2.530338        | 2.966744                  | 0.1699   |

```
In [20]: fig, ax = plt.subplots(figsize=(10, 6))

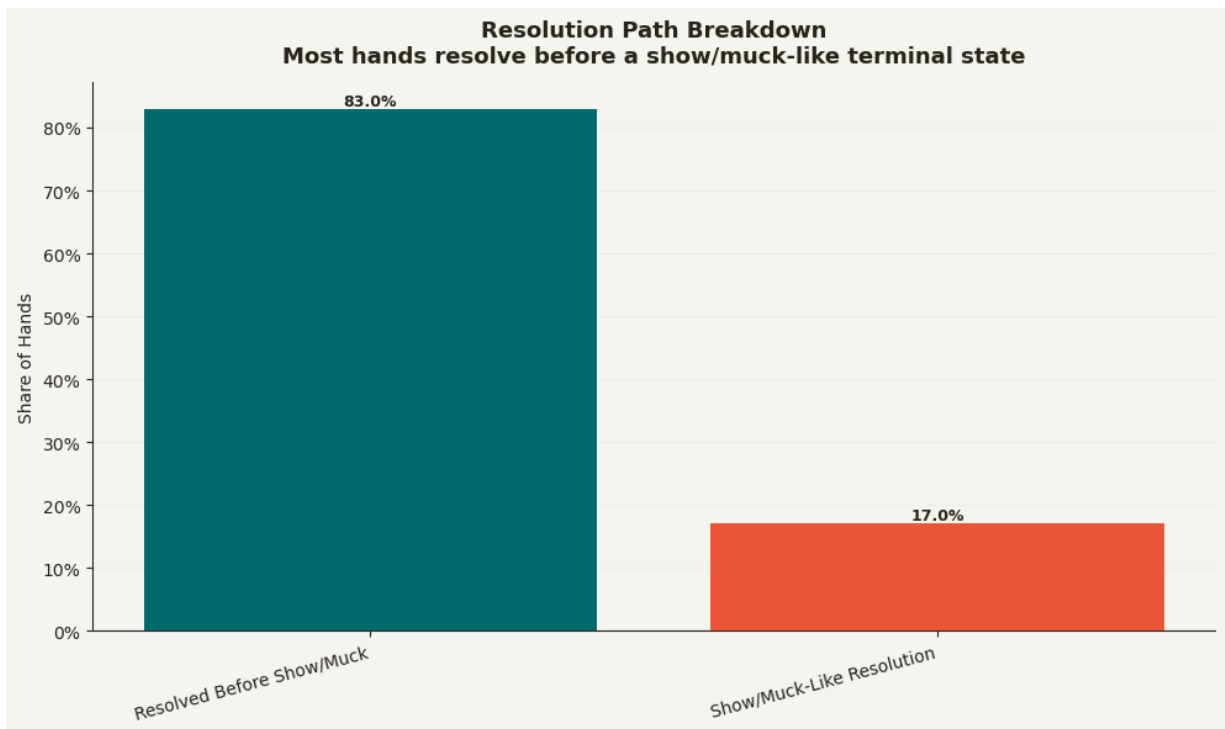
bars = ax.bar(
    resolution_summary["resolution_path"],
    resolution_summary["hand_share"] * 100,
    color=[TEAL, RED, GRAY][:len(resolution_summary)],
    zorder=3,
)

for bar in bars:
    height = bar.get_height()
    ax.text(
        bar.get_x() + bar.get_width() / 2,
        height,
        f"{height:.1f}%",
        ha="center",
        va="bottom",
        fontsize=9,
        fontweight="bold",
    )

ax.set_title(
    "Resolution Path Breakdown\n"
    "Most hands resolve before a show/muck-like terminal state",
    fontsize=13,
    fontweight="bold",
    pad=12,
)

ax.set_xlabel("")
ax.set_ylabel("Share of Hands")
ax.yaxis.set_major_formatter(mtick.PercentFormatter())
ax.grid(axis="y", alpha=0.25, color=GRID, zorder=0)
ax.spines[["top", "right"]].set_visible(False)

plt.xticks(rotation=15, ha="right")
plt.tight_layout()
plt.savefig("phh_resolution_path_breakdown.png", dpi=150, bbox_inches="tight", face
plt.show()
```



## Interpretation: Resolution Paths

Most hands resolve before show/muck-like activity. A smaller share reaches a more visible terminal state, but the later visibility checks show that even show/muck-like resolution does not guarantee exposed cards in this PHH corpus.

**\*\*Commercial read:\*\*** resolution path is useful for understanding experience clarity and contest pacing, but it should not be treated as a complete card-disclosure metric.

## Contest Outcome Intensity

The next section shifts away from card strength and toward parsed economic outcomes. Winner-side card visibility is too limited for strategy conclusions, but stack and outcome data can still help describe which hands create larger observable value swings.

## Outcome Parser

Winner-side outcome extraction uses the reusable helper layer. It identifies positive stack changes where available, checks winner-card visibility, and carries forward the same action metrics used earlier in the notebook.

```
In [21]: winner_rows = [parse_winner_outcome(path) for path in tqdm(phh_files, desc="Parsing  
winner_df = pd.DataFrame(winner_rows)
```

```
print(winner_df.shape)
display(winner_df.head())
```

```
Parsing winner-side hand data: 100%|████████████████████████████████████████| 10
088/10088 [00:10<00:00, 992.33it/s]
(10088, 31)
```

|   | file                                              | filename                  | parse_success | variant | currency | year   |    |
|---|---------------------------------------------------|---------------------------|---------------|---------|----------|--------|----|
| 0 | C:\Users\J1999\Downloads\poker-hand-histories\... | alice-carol-wikipedia.phh | True          | FB      | None     | NaN    | l  |
| 1 | C:\Users\J1999\Downloads\poker-hand-histories\... | antonius-blom-2009.phh    | True          | PO      | USD      | 2009.0 | v  |
| 2 | C:\Users\J1999\Downloads\poker-hand-histories\... | arieh-yockey-2019.phh     | True          | F2L3D   | None     | 2019.0 | \  |
| 3 | C:\Users\J1999\Downloads\poker-hand-histories\... | dwan-ivey-2009.phh        | True          | NT      | USD      | 2009.0 | .  |
| 4 | C:\Users\J1999\Downloads\poker-hand-histories\... | phua-xuan-2019.phh        | True          | NS      | GBP      | 2019.0 | Ba |

5 rows  $\times$  31 columns

```
In [22]: print("Parse success:")
display(winner_df["parse_success"].value_counts(dropna=False))

print("\nNumber of winners per hand:")
display(winner_df["num_winners"].value_counts(dropna=False).sort_index())

print("\nWinner known-card visibility:")
display(winner_df["winner_has_known_cards"].value_counts(normalize=True, dropna=False))

print("\nHand known-card visibility:")
display(winner_df["hand_has_known_cards"].value_counts(normalize=True, dropna=False))

print("\nCore winner-side metrics:")
display(winner_df[[
    "num_actions",
    "num_bets_raises",
    "bet_raise_rate",
    "num_showdown_like",
    "winner_known_card_count",
```

```
"hand_known_card_count"
]].describe())
```

```
Parse success:
parse_success
True      10088
Name: count, dtype: int64
Number of winners per hand:
num_winners
0         41
1      9991
2         56
Name: count, dtype: int64
Winner known-card visibility:
winner_has_known_cards
False      0.997224
True       0.002776
Name: proportion, dtype: float64
Hand known-card visibility:
hand_has_known_cards
False      0.612411
True       0.387589
Name: proportion, dtype: float64
Core winner-side metrics:
```

|              | num_actions  | num_bets_raises | bet_raise_rate | num_showdown_like | winner_known_cards |
|--------------|--------------|-----------------|----------------|-------------------|--------------------|
| <b>count</b> | 10088.000000 | 10088.000000    | 10088.000000   | 10088.000000      | 10088              |
| <b>mean</b>  | 16.688243    | 1.857950        | 0.105152       | 0.349921          | 0                  |
| <b>std</b>   | 5.057247     | 1.199759        | 0.055205       | 0.793474          | 0                  |
| <b>min</b>   | 5.000000     | 0.000000        | 0.000000       | 0.000000          | 0                  |
| <b>25%</b>   | 12.000000    | 1.000000        | 0.083333       | 0.000000          | 0                  |
| <b>50%</b>   | 16.000000    | 2.000000        | 0.095238       | 0.000000          | 0                  |
| <b>75%</b>   | 22.000000    | 3.000000        | 0.142857       | 0.000000          | 0                  |
| <b>max</b>   | 36.000000    | 7.000000        | 0.437500       | 10.000000         | 8                  |

```
In [23]: winner_value_df = winner_df[winner_df["parse_success"].eq(True)].copy()

def max_positive_net(net_changes):
    if isinstance(net_changes, list) and len(net_changes) > 0:
        vals = [x for x in net_changes if isinstance(x, (int, float))]
        positives = [x for x in vals if x > 0]
        return max(positives) if positives else np.nan
    return np.nan

def total_positive_net(net_changes):
    if isinstance(net_changes, list) and len(net_changes) > 0:
        vals = [x for x in net_changes if isinstance(x, (int, float))]
        positives = [x for x in vals if x > 0]
        return sum(positives) if positives else np.nan
    return np.nan
```

```

winner_value_df["max_winner_net"] = winner_value_df["net_changes"].apply(max_positi
winner_value_df["total_positive_net"] = winner_value_df["net_changes"].apply(total_

winner_value_df["has_showdown"] = winner_value_df["showdown_like_hand"]
winner_value_df["actions_per_player"] = winner_value_df["num_actions"] / winner_val

winner_value_df[[
    "num_actions",
    "num_bets_raises",
    "bet_raise_rate",
    "showdown_like_hand",
    "max_winner_net",
    "total_positive_net"
]].describe()

```

Out[23]:

|              | num_actions  | num_bets_raises | bet_raise_rate | max_winner_net | total_positive_net |
|--------------|--------------|-----------------|----------------|----------------|--------------------|
| <b>count</b> | 10088.000000 | 10088.000000    | 10088.000000   | 1.004700e+04   | 1.004700e+04       |
| <b>mean</b>  | 16.688243    | 1.857950        | 0.105152       | 8.350668e+03   | 8.394548e+03       |
| <b>std</b>   | 5.057247     | 1.199759        | 0.055205       | 1.143900e+05   | 1.144417e+05       |
| <b>min</b>   | 5.000000     | 0.000000        | 0.000000       | 2.500000e+01   | 5.000000e+01       |
| <b>25%</b>   | 12.000000    | 1.000000        | 0.083333       | 1.500000e+02   | 1.500000e+02       |
| <b>50%</b>   | 16.000000    | 2.000000        | 0.095238       | 2.750000e+02   | 2.750000e+02       |
| <b>75%</b>   | 22.000000    | 3.000000        | 0.142857       | 6.000000e+02   | 6.000000e+02       |
| <b>max</b>   | 36.000000    | 7.000000        | 0.437500       | 3.750000e+06   | 3.750000e+06       |

In [24]:

```

plot_df = winner_value_df.dropna(subset=["max_winner_net"]).copy()

# Remove extreme outliers for readable chart
upper = plot_df["max_winner_net"].quantile(0.99)
plot_trim = plot_df[plot_df["max_winner_net"] <= upper].copy()

fig, ax = plt.subplots(figsize=(12, 6))

ax.hist(
    plot_trim["max_winner_net"],
    bins=40,
    color=TEAL,
    edgecolor=TEXT,
    alpha=0.85,
    zorder=3
)

ax.axvline(
    plot_trim["max_winner_net"].median(),
    color=RED,
    linestyle="--",
    linewidth=2,
    label=f"Median: {plot_trim['max_winner_net'].median():.0f}"
)

```

```

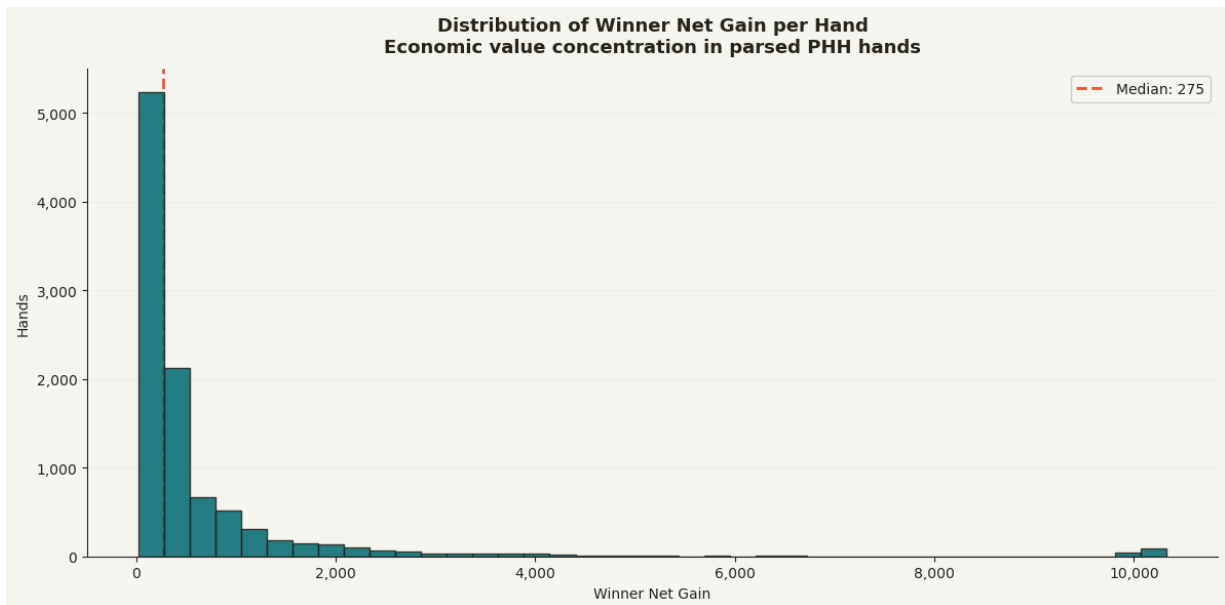
)

ax.set_title(
    "Distribution of Winner Net Gain per Hand\n"
    "Economic value concentration in parsed PHH hands",
    fontsize=13,
    fontweight="bold",
    pad=12
)

ax.set_xlabel("Winner Net Gain")
ax.set_ylabel("Hands")
ax.yaxis.set_major_formatter(mtick.StrMethodFormatter("{x:,.0f}"))
ax.xaxis.set_major_formatter(mtick.StrMethodFormatter("{x:,.0f}"))
ax.grid(axis="y", alpha=0.25, color=GRID, zorder=0)
ax.legend(framealpha=0.9)
ax.spines[["top", "right"]].set_visible(False)

plt.tight_layout()
plt.savefig("phh_winner_net_gain_distribution.png", dpi=150, bbox_inches="tight", f
plt.show()

```



```

In [25]: plot_df = winner_value_df.dropna(subset=["num_actions", "max_winner_net"]).copy()

# Trim extreme winner-net outliers for readability
upper = plot_df["max_winner_net"].quantile(0.99)
plot_trim = plot_df[plot_df["max_winner_net"] <= upper].copy()

action_value_summary = (
    plot_trim
    .groupby("num_actions", as_index=False)
    .agg(
        hands=("file", "count"),
        avg_winner_net=("max_winner_net", "mean"),
        median_winner_net=("max_winner_net", "median"),
        avg_bets_raises=("num_bets_raises", "mean"),
        showdown_rate=("showdown_like_hand", "mean")
    )

```

```

    )
)

# Keep action counts with enough hands
action_value_summary = action_value_summary[action_value_summary["hands"] >= 20]

fig, ax = plt.subplots(figsize=(12, 6))

ax.plot(
    action_value_summary["num_actions"],
    action_value_summary["median_winner_net"],
    marker="o",
    linewidth=2.5,
    color=TEAL,
    label="Median Winner Net Gain",
    zorder=3
)

ax.plot(
    action_value_summary["num_actions"],
    action_value_summary["avg_winner_net"],
    marker="o",
    linewidth=2.5,
    color=RED,
    label="Avg Winner Net Gain",
    zorder=3
)

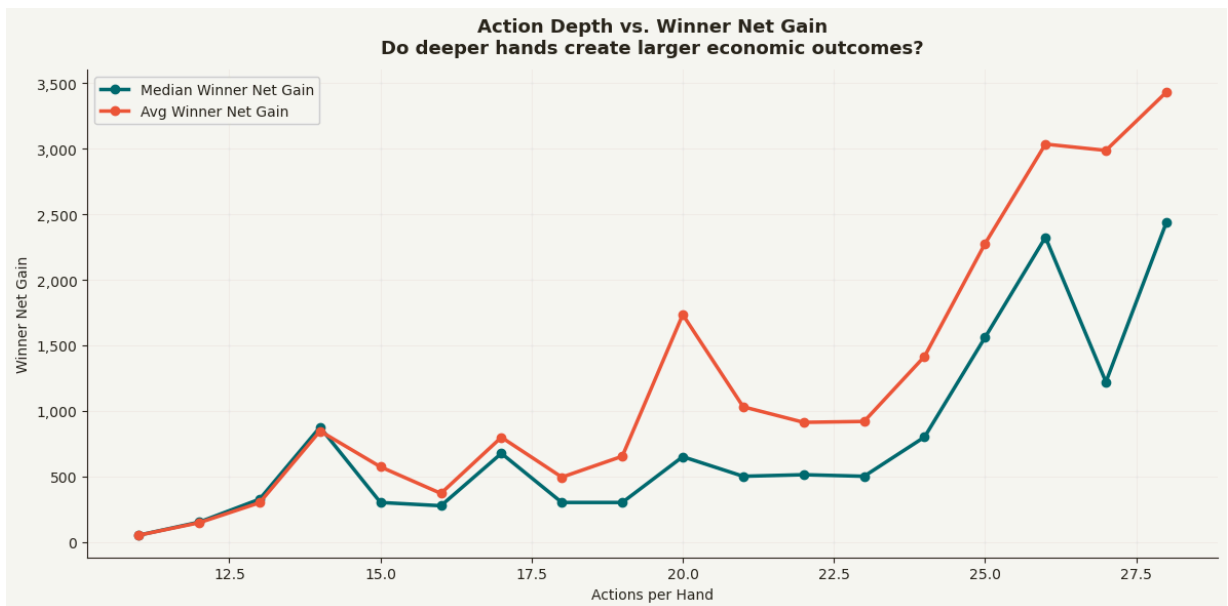
ax.set_title(
    "Action Depth vs. Winner Net Gain\n"
    "Do deeper hands create larger economic outcomes?",
    fontsize=13,
    fontweight="bold",
    pad=12
)

ax.set_xlabel("Actions per Hand")
ax.set_ylabel("Winner Net Gain")
ax.yaxis.set_major_formatter(mtick.StrMethodFormatter("{x:,.0f}"))
ax.grid(alpha=0.25, color=GRID, zorder=0)
ax.legend(framealpha=0.9)
ax.spines[["top", "right"]].set_visible(False)

plt.tight_layout()
plt.savefig("phh_action_depth_vs_winner_net_gain.png", dpi=150, bbox_inches="tight")
plt.show()

action_value_summary.head()

```



Out[25]:

|   | num_actions | hands | avg_winner_net | median_winner_net | avg_bets_raises | showdown_rat |
|---|-------------|-------|----------------|-------------------|-----------------|--------------|
| 0 | 11          | 1058  | 50.000000      | 50.0              | 0.000000        | 0.           |
| 1 | 12          | 2522  | 145.459952     | 150.0             | 1.000000        | 0.           |
| 2 | 13          | 847   | 298.060213     | 325.0             | 1.866588        | 0.           |
| 3 | 14          | 200   | 845.470000     | 875.0             | 2.685000        | 0.           |
| 4 | 15          | 187   | 571.962567     | 300.0             | 2.005348        | 0.           |

```
In [26]: showdown_value_summary = (
    winner_value_df
    .dropna(subset=["max_winner_net"])
    .groupby("showdown_like_hand", as_index=False)
    .agg(
        hands=("file", "count"),
        avg_actions=("num_actions", "mean"),
        avg_bets_raises=("num_bets_raises", "mean"),
        median_winner_net=("max_winner_net", "median"),
        avg_winner_net=("max_winner_net", "mean"),
        avg_bet_raise_rate=("bet_raise_rate", "mean")
    )
)

showdown_value_summary["label"] = np.where(
    showdown_value_summary["showdown_like_hand"],
    "Showdown-Like Hands",
    "Non-Showdown Hands"
)

fig, ax = plt.subplots(figsize=(10, 6))

x = np.arange(len(showdown_value_summary))
width = 0.36

ax.bar(
```



```

        x - width / 2,
        showdown_value_summary["avg_actions"],
        width,
        label="Avg Actions",
        color=TEAL,
        zorder=3
    )

ax.bar(
    x + width / 2,
    showdown_value_summary["avg_bets_raises"],
    width,
    label="Avg Bets/Raises",
    color=RED,
    zorder=3
)

ax.set_xticks(x)
ax.set_xticklabels(showdown_value_summary["label"])

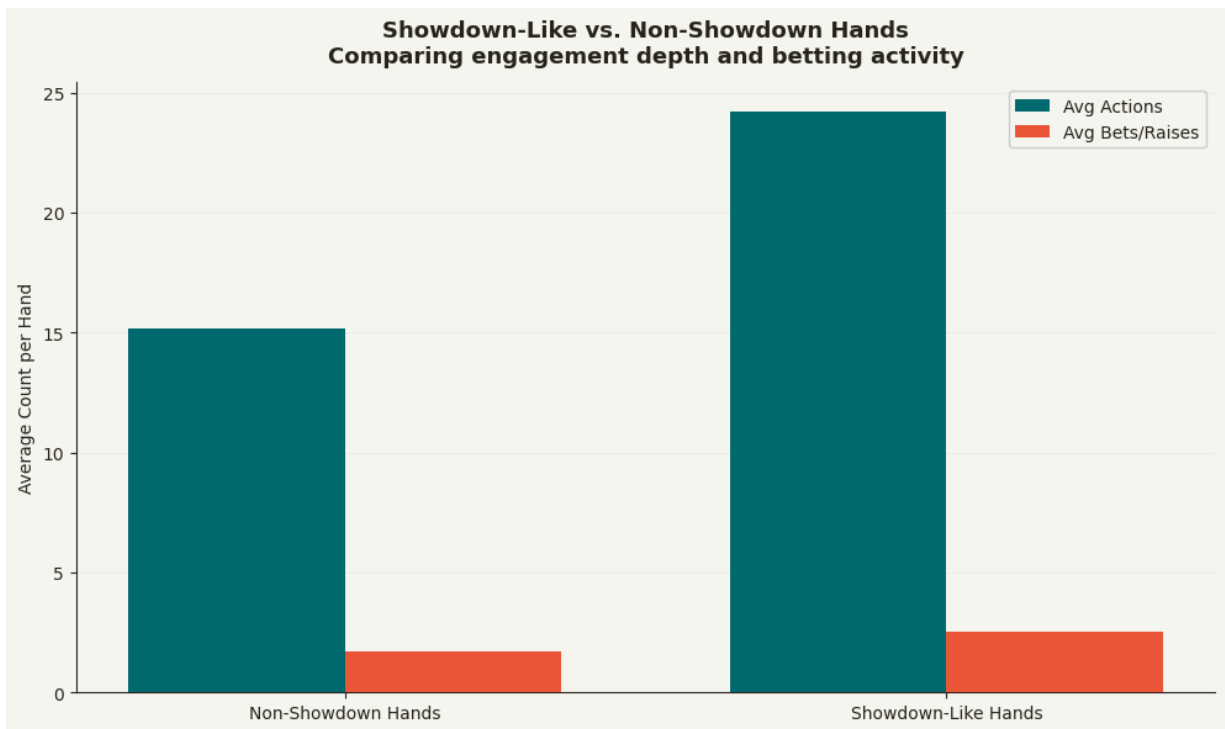
ax.set_title(
    "Showdown-Like vs. Non-Showdown Hands\n"
    "Comparing engagement depth and betting activity",
    fontsize=13,
    fontweight="bold",
    pad=12
)

ax.set_ylabel("Average Count per Hand")
ax.grid(axis="y", alpha=0.25, color=GRID, zorder=0)
ax.legend(framealpha=0.9)
ax.spines[["top", "right"]].set_visible(False)

plt.tight_layout()
plt.savefig("phh_showdown_vs_non_showdown_engagement.png", dpi=150, bbox_inches="tight")
plt.show()

showdown_value_summary

```



Out[26]:

|   | showdown_like_hand | hands | avg_actions | avg_bets_raises | median_winner_net | avg_winn |
|---|--------------------|-------|-------------|-----------------|-------------------|----------|
| 0 | False              | 8374  | 15.152615   | 1.720325        | 250.0             | 3280.6   |
| 1 | True               | 1673  | 24.202630   | 2.536760        | 819.0             | 33727.9  |

## Interpretation: Outcome Intensity

Outcome-size metrics should be read as contest dynamics, not as proof of player quality. The useful business question is whether deeper or showdown-like hands tend to produce larger observable value swings and more involved gameplay experiences.

**\*\*Commercial read:\*\*** operators care about the distribution of experience intensity. A healthy ecosystem may need both quick, accessible interactions and higher-intensity moments that create excitement and retention hooks.

## Normalized Outcome Intensity

Raw winner net gain can be misleading when sources use different stakes or chip scales. This section normalizes outcome size by total starting stack so the metric reads as a share of available contest value.

This is a better commercial measure of **\*\*value swing intensity\*\*** than raw chip or currency amounts.

```

In [27]: winner_value_df["total_starting_stack"] = winner_value_df["starting_stacks"].apply(
    lambda stacks: sum(x for x in stacks if isinstance(x, (int, float)))
    if isinstance(stacks, list)
    else np.nan
)

winner_value_df["winner_net_gain_pct_of_starting_stack"] = (
    winner_value_df["max_winner_net"] / winner_value_df["total_starting_stack"]
)

outcome_intensity_df = winner_value_df.dropna(
    subset=["winner_net_gain_pct_of_starting_stack", "max_winner_net"]
).copy()

# Use Pluribus as the main comparable population because it represents nearly all h
# and has a consistent six-player no-limit format.
outcome_intensity_df["source_folder"] = outcome_intensity_df["file"].apply(lambda x
pluribus_outcome_df = outcome_intensity_df[outcome_intensity_df["source_folder"].eq

outcome_intensity_summary = pd.DataFrame([
    "hands": len(pluribus_outcome_df),
    "median_outcome_intensity": pluribus_outcome_df["winner_net_gain_pct_of_startin
    "p75_outcome_intensity": pluribus_outcome_df["winner_net_gain_pct_of_starting_s
    "p90_outcome_intensity": pluribus_outcome_df["winner_net_gain_pct_of_starting_s
    "p95_outcome_intensity": pluribus_outcome_df["winner_net_gain_pct_of_starting_s
    "p99_outcome_intensity": pluribus_outcome_df["winner_net_gain_pct_of_starting_s
]])

outcome_intensity_summary.T

```

Out[27]: **0**

|                                 | hands       |
|---------------------------------|-------------|
|                                 | 9966.000000 |
| <b>median_outcome_intensity</b> | 0.004583    |
| <b>p75_outcome_intensity</b>    | 0.009583    |
| <b>p90_outcome_intensity</b>    | 0.023975    |
| <b>p95_outcome_intensity</b>    | 0.041667    |
| <b>p99_outcome_intensity</b>    | 0.168333    |

```

In [28]: plot_df = pluribus_outcome_df.copy()
upper = plot_df["winner_net_gain_pct_of_starting_stack"].quantile(0.99)
plot_trim = plot_df[plot_df["winner_net_gain_pct_of_starting_stack"] <= upper].copy

fig, ax = plt.subplots(figsize=(12, 6))

ax.hist(
    plot_trim["winner_net_gain_pct_of_starting_stack"] * 100,
    bins=40,
    color=TEAL,
    edgecolor=TEXT,

```

```

    alpha=0.85,
    zorder=3,
)

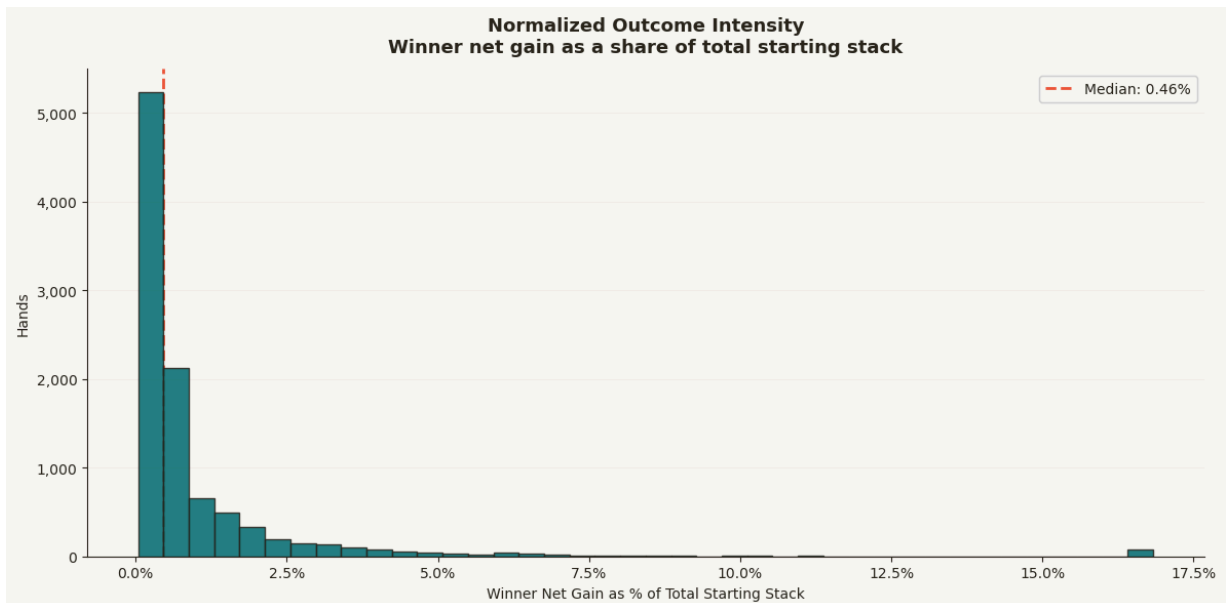
ax.axvline(
    plot_trim["winner_net_gain_pct_of_starting_stack"].median() * 100,
    color=RED,
    linestyle="--",
    linewidth=2,
    label=f"Median: {plot_trim['winner_net_gain_pct_of_starting_stack'].median() *
)

ax.set_title(
    "Normalized Outcome Intensity\n"
    "Winner net gain as a share of total starting stack",
    fontsize=13,
    fontweight="bold",
    pad=12,
)

ax.set_xlabel("Winner Net Gain as % of Total Starting Stack")
ax.set_ylabel("Hands")
ax.xaxis.set_major_formatter(mtick.PercentFormatter())
ax.yaxis.set_major_formatter(mtick.StrMethodFormatter("{x:,.0f}"))
ax.grid(axis="y", alpha=0.25, color=GRID, zorder=0)
ax.legend(framealpha=0.9)
ax.spines[["top", "right"]].set_visible(False)

plt.tight_layout()
plt.savefig("phh_normalized_outcome_intensity.png", dpi=150, bbox_inches="tight", f
plt.show()

```



## Interpretation: Normalized Outcome Intensity

Normalizing by total starting stack makes the outcome metric comparable within the main Pluribus population. It reframes

the question from “how many chips changed hands?” to “how large was the value swing relative to the contest environment?”

**\*\*Commercial read:\*\*** this is closer to a volatility or intensity KPI. It can help identify whether engagement is concentrated in a small number of high-swing moments.

## Outcome Concentration

A product analytics team may care not only about the average hand, but also whether a small share of interactions creates a large share of the economic intensity. This is similar to concentration analysis in marketplaces, contests, and engagement ecosystems.

```
In [29]: def concentration_share(series, top_share):
          values = series.dropna().sort_values(ascending=False).reset_index(drop=True)
          if len(values) == 0 or values.sum() == 0:
              return np.nan
          n = max(1, int(len(values) * top_share))
          return values.iloc[:n].sum() / values.sum()

concentration_summary = pd.DataFrame([
    {
        "top_hand_share": "Top 1%",
        "outcome_intensity_share": concentration_share(
            pluribus_outcome_df["winner_net_gain_pct_of_starting_stack"], 0.01
        ),
    },
    {
        "top_hand_share": "Top 5%",
        "outcome_intensity_share": concentration_share(
            pluribus_outcome_df["winner_net_gain_pct_of_starting_stack"], 0.05
        ),
    },
    {
        "top_hand_share": "Top 10%",
        "outcome_intensity_share": concentration_share(
            pluribus_outcome_df["winner_net_gain_pct_of_starting_stack"], 0.10
        ),
    },
    {
        "top_hand_share": "Top 20%",
        "outcome_intensity_share": concentration_share(
            pluribus_outcome_df["winner_net_gain_pct_of_starting_stack"], 0.20
        ),
    },
])

concentration_summary
```

Out[29]:

|   | top_hand_share | outcome_intensity_share |
|---|----------------|-------------------------|
| 0 | Top 1%         | 0.150327                |
| 1 | Top 5%         | 0.421274                |
| 2 | Top 10%        | 0.558974                |
| 3 | Top 20%        | 0.710462                |

|   | top_hand_share | outcome_intensity_share |
|---|----------------|-------------------------|
| 0 | Top 1%         | 0.150327                |
| 1 | Top 5%         | 0.421274                |
| 2 | Top 10%        | 0.558974                |
| 3 | Top 20%        | 0.710462                |

```
In [30]: fig, ax = plt.subplots(figsize=(10, 6))

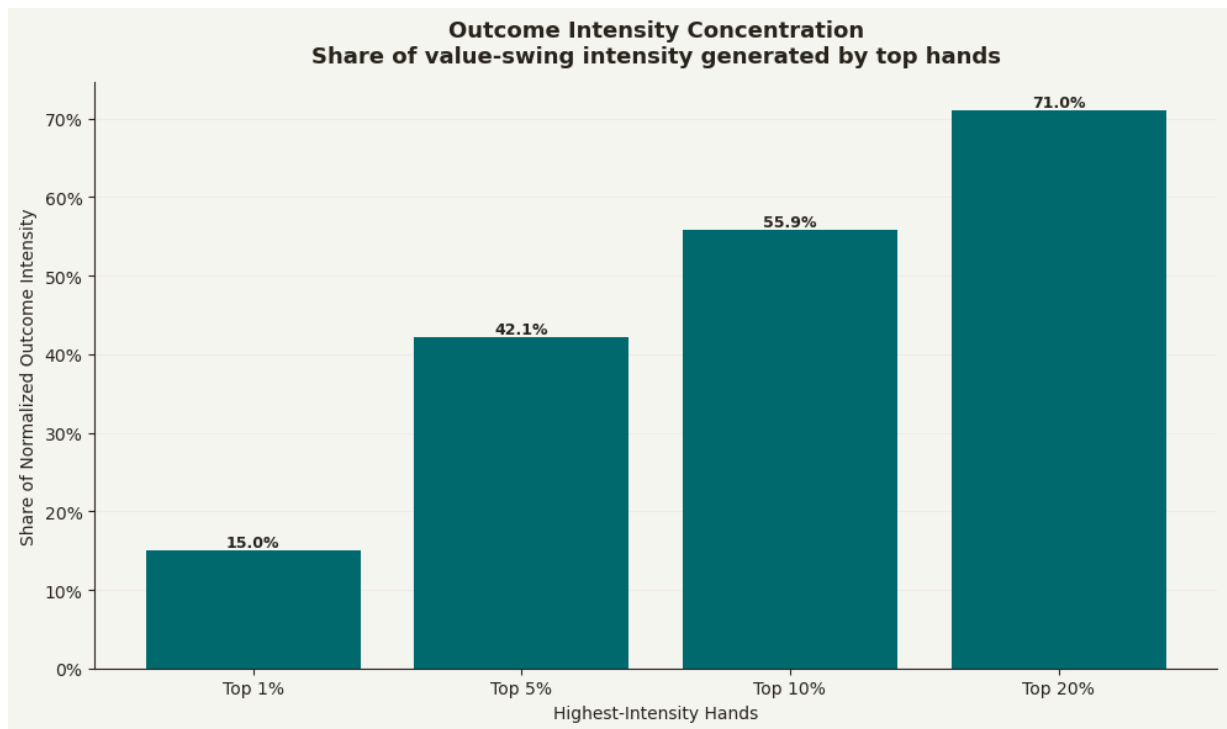
bars = ax.bar(
    concentration_summary["top_hand_share"],
    concentration_summary["outcome_intensity_share"] * 100,
    color=TEAL,
    zorder=3,
)

for bar in bars:
    height = bar.get_height()
    ax.text(
        bar.get_x() + bar.get_width() / 2,
        height,
        f"{height:.1f}%",
        ha="center",
        va="bottom",
        fontsize=9,
        fontweight="bold",
    )

ax.set_title(
    "Outcome Intensity Concentration\n"
    "Share of value-swing intensity generated by top hands",
    fontsize=13,
    fontweight="bold",
    pad=12,
)

ax.set_xlabel("Highest-Intensity Hands")
ax.set_ylabel("Share of Normalized Outcome Intensity")
ax.yaxis.set_major_formatter(mtick.PercentFormatter())
ax.grid(axis="y", alpha=0.25, color=GRID, zorder=0)
ax.spines[["top", "right"]].set_visible(False)

plt.tight_layout()
plt.savefig("phh_outcome_intensity_concentration.png", dpi=150, bbox_inches="tight")
plt.show()
```



## Interpretation: Outcome Concentration

A relatively small share of hands accounts for a large share of normalized outcome intensity. That does not mean those hands are better or more skillful; it means the product experience has a long-tail intensity pattern.

**\*\*Commercial read:\*\*** this resembles marketplace or contest ecosystem analysis, where a small number of events can drive a disproportionate share of activity, volatility, or memorable user experience.

## Anonymized Participant Behavior Profiles

This section demonstrates how the same event-log workflow could support participant segmentation. The goal is not to rank poker skill. The goal is to show how observable actions can be converted into behavioral profiles.

Because this dataset contains named example participants, the display below anonymizes them before presentation.

```
In [32]: participant_rows = []

for path in tqdm([p for p in phh_files if source_folder(p) == "pluribus"], desc="Pr
try:
    participant_rows.extend(parse_participant_rows(path))
except Exception:
    pass
```





Out[32]:

|    | participant    | hands | active_rate | avg_actions | high_commitment_action_rate | fold_hand_ra |
|----|----------------|-------|-------------|-------------|-----------------------------|--------------|
| 3  | Participant 04 | 976   | 0.973       | 1.713       | 0.260                       | 0.7          |
| 7  | Participant 08 | 18242 | 0.979       | 1.710       | 0.234                       | 0.7          |
| 0  | Participant 01 | 13426 | 0.985       | 1.603       | 0.234                       | 0.7          |
| 2  | Participant 03 | 11020 | 0.983       | 1.616       | 0.220                       | 0.7          |
| 4  | Participant 05 | 2730  | 0.974       | 1.596       | 0.216                       | 0.7          |
| 13 | Participant 14 | 20000 | 0.984       | 1.574       | 0.214                       | 0.8          |
| 10 | Participant 11 | 12110 | 0.982       | 1.566       | 0.214                       | 0.8          |
| 12 | Participant 13 | 1542  | 0.981       | 1.560       | 0.211                       | 0.8          |
| 1  | Participant 02 | 5018  | 0.984       | 1.510       | 0.211                       | 0.8          |
| 5  | Participant 06 | 3070  | 0.991       | 1.484       | 0.205                       | 0.8          |
| 11 | Participant 12 | 8966  | 0.982       | 1.567       | 0.202                       | 0.8          |
| 8  | Participant 09 | 2756  | 0.979       | 1.571       | 0.200                       | 0.8          |
| 6  | Participant 07 | 5120  | 0.980       | 1.432       | 0.188                       | 0.8          |
| 9  | Participant 10 | 15024 | 0.985       | 1.481       | 0.169                       | 0.8          |

## Interpretation: Participant Profiles

This table is an example of behavioral segmentation from event logs. It identifies differences in observable participation, high-commitment action frequency, folding frequency, and outcomes.

**\*\*Commercial read:\*\*** in a real gaming analytics environment, this profile could be joined to tenure, acquisition channel, session frequency, deposits, contest type, promotions, or retention outcomes. In this public PHH dataset, it should remain descriptive rather than causal.

# Explicit Obfuscation Test

This validation section checks whether show/muck-style resolution actually exposes cards. It is intentionally included to prevent overclaiming.

```
In [34]: obf_rows = [
          parse_obfuscation_resolution(path)
          for path in tqdm(phh_files, desc="Checking obfuscation vs resolution")
        ]
obf_df = pd.DataFrame(obf_rows)

print(obf_df.shape)
display(obf_df.head())
```

Checking obfuscation vs resolution: 100% | 10088/10088 [00:07<00:00, 1402.19it/s] (10088, 23)

|   | file                                              | filename                  | parse_success | variant | num_players | num_a |
|---|---------------------------------------------------|---------------------------|---------------|---------|-------------|-------|
| 0 | C:\Users\J1999\Downloads\poker-hand-histories\... | alice-carol-wikipedia.phh | True          | FB      | 4           |       |
| 1 | C:\Users\J1999\Downloads\poker-hand-histories\... | antonius-blom-2009.phh    | True          | PO      | 2           |       |
| 2 | C:\Users\J1999\Downloads\poker-hand-histories\... | arieh-yockey-2019.phh     | True          | F2L3D   | 4           |       |
| 3 | C:\Users\J1999\Downloads\poker-hand-histories\... | dwan-ivey-2009.phh        | True          | NT      | 3           |       |
| 4 | C:\Users\J1999\Downloads\poker-hand-histories\... | phua-xuan-2019.phh        | True          | NS      | 6           |       |

5 rows × 23 columns

```
In [35]: print("Parse success:")
display(obf_df["parse_success"].value_counts(dropna=False))

print("\nResolution type counts:")
resolution_summary = (
    obf_df
    .groupby("resolution_type", as_index=False)
    .agg(hands=("file", "count"))
    .sort_values("hands", ascending=False)
)
```

```

resolution_summary["share"] = resolution_summary["hands"] / resolution_summary["hands"]
display(resolution_summary)

print("\nShow/muck card visibility summary:")
sm_summary = pd.DataFrame([
    "total_hands": len(obf_df),
    "hands_with_show_muck": int(obf_df["has_sm"].sum()),
    "hands_with_known_show_muck_cards": int((obf_df["sm_known_actions"] > 0).sum()),
    "hands_with_obfuscated_show_muck_cards": int((obf_df["sm_obfuscated_actions"] > 0).sum()),
    "hands_with_show_muck_but_no_card_text": int((obf_df["sm_no_card_actions"] > 0).sum()),
    "hands_with_any_obfuscation": int(obf_df["has_any_obfuscation"].sum()),
    "hands_with_known_cards_anywhere": int(obf_df["has_known_cards"].sum()),
])

display(sm_summary.T)

print("\nLast non-deal action codes:")
display(obf_df["last_non_deal_code"].value_counts(dropna=False).head(20))

```

Parse success:

parse\_success

True 10088

Name: count, dtype: int64

Resolution type counts:

|   | resolution_type             | hands | share    |
|---|-----------------------------|-------|----------|
| 0 | No show/muck activity       | 8374  | 0.830095 |
| 1 | Show/muck with no card text | 1714  | 0.169905 |

Show/muck card visibility summary:

|                                       |       |
|---------------------------------------|-------|
|                                       | 0     |
| total_hands                           | 10088 |
| hands_with_show_muck                  | 1714  |
| hands_with_known_show_muck_cards      | 0     |
| hands_with_obfuscated_show_muck_cards | 0     |
| hands_with_show_muck_but_no_card_text | 1714  |
| hands_with_any_obfuscation            | 19    |
| hands_with_known_cards_anywhere       | 3910  |

Last non-deal action codes:

last\_non\_deal\_code

f 8374

sm 1714

Name: count, dtype: int64

```

In [36]: no_sm = (obf_df["resolution_type"] == "No show/muck activity").sum()
sm_obf = (obf_df["resolution_type"] == "Show/muck with obfuscated cards").sum()
sm_known = (obf_df["resolution_type"] == "Show/muck with known cards").sum()
sm_no_text = (obf_df["resolution_type"] == "Show/muck with no card text").sum()

```

```

print(f"No show/muck activity: {no_sm:,} hands ({no_sm / len(obf_df):.1%})")
print(f"Show/muck with known cards: {sm_known:,} hands ({sm_known / len(obf_df):.1%})")
print(f"Show/muck with obfuscated cards: {sm_obf:,} hands ({sm_obf / len(obf_df):.1%})")
print(f"Show/muck with no card text: {sm_no_text:,} hands ({sm_no_text / len(obf_df):.1%})")

if no_sm > sm_obf:
    print("\nMain issue: most hands simply do not have show/muck-style resolution activity.")
else:
    print("\nMain issue: show/muck happens, but card information is mostly obfuscated.")

```

```

No show/muck activity: 8,374 hands (83.0%)
Show/muck with known cards: 0 hands (0.0%)
Show/muck with obfuscated cards: 0 hands (0.0%)
Show/muck with no card text: 1,714 hands (17.0%)

```

Main issue: most hands simply do not have show/muck-style resolution activity.

## Commercial Takeaways

- Raw gameplay logs can be transformed into business-readable metrics for engagement depth, commitment intensity, lifecycle depth, resolution visibility, and outcome concentration.
- The strongest analysis is action-based because observable actions are more reliable than private-card visibility in this PHH corpus.
- Private-card and winner-card analysis should not be used for strategy claims because visibility is limited and partially obfuscated.
- Action-based segments create a practical framework for monitoring product experience across formats, cohorts, or time periods.
- Contest lifecycle depth adds a funnel-style view of how far interactions progress before resolving.
- Normalized outcome intensity avoids overreading raw chip values and shows whether value swings are concentrated in a small share of hands.
- Participant profiles show how event logs can support behavioral segmentation, while remaining descriptive rather than causal.
- The project demonstrates a scalable analytics workflow for imperfect event data: audit coverage, define observable metrics, test data limitations, and communicate conclusions with appropriate caution.

## Recommended Next Steps

With richer internal gaming data, this workflow could be extended by joining hand-level behavior to player tenure, session frequency, buy-in level, contest type, acquisition channel, promotion exposure, and retention outcomes. That would turn the notebook from exploratory product analytics into a stronger commercial intelligence workflow.